

Programa

Sugeneruota Doxygen 1.16.1

1 Direktorijų hierarchija	1
1.1 Direktorijos	1
2 Hierarchijos Indeksas	3
2.1 Klasių hierarchija	3
3 Klasės Indeksas	5
3.1 Klasės	5
4 Failo Indeksas	7
4.1 Failai	7
5 Direktorijų dokumentacija	9
5.1 src Direktorijos aprašas	9
6 Klasės Dokumentacija	11
6.1 Programos_laikai Klasė	11
6.1.1 Metodų Dokumentacija	11
6.1.1.1 spausd_laikus()	11
6.1.2 Atributų Dokumentacija	11
6.1.2.1 duomenu_nuskaitymas	11
6.1.2.2 duomenu_rikiavimas	11
6.1.2.3 studentu_skirstymas	12
6.1.2.4 visa_trukme_su_ivestim	12
6.2 Studentas Klasė	12
6.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	14
6.2.1.1 Studentas() [1/4]	14
6.2.1.2 Studentas() [2/4]	14
6.2.1.3 Studentas() [3/4]	15
6.2.1.4 Studentas() [4/4]	15
6.2.1.5 ~Studentas()	15
6.2.2 Metodų Dokumentacija	15
6.2.2.1 apsk_med()	15
6.2.2.2 apsk_vid()	15
6.2.2.3 Container()	16
6.2.2.4 egzo_rezas()	16
6.2.2.5 galut()	16
6.2.2.6 ivest_egzo_reza()	16
6.2.2.7 ivest_varda_pavarde()	16
6.2.2.8 nust_egzo_reza()	17
6.2.2.9 nust_galutinio_tipa()	17
6.2.2.10 operator=() [1/2]	17
6.2.2.11 operator=() [2/2]	17
6.2.2.12 pazymiu_sk()	18

6.2.2.13 pridet_pazymi()	18
6.2.2.14 rezas_med()	18
6.2.2.15 rezas_vid()	18
6.2.2.16 uzpildyt_pazymius_iki_min()	19
6.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	19
6.2.3.1 operator<<	19
6.2.3.2 operator>>	19
6.3 Vector< T > Klasė Šablonas	20
6.3.1 Tipo Aprašymo Dokumentacija	22
6.3.1.1 const_iterator	22
6.3.1.2 const_pointer	22
6.3.1.3 const_reference	22
6.3.1.4 const_reverse_iterator	22
6.3.1.5 difference_type	22
6.3.1.6 iterator	23
6.3.1.7 pointer	23
6.3.1.8 reference	23
6.3.1.9 reverse_iterator	23
6.3.1.10 size_type	23
6.3.1.11 value_type	23
6.3.2 Konstruktorius ir Destruktorius Dokumentacija	23
6.3.2.1 Vector() [1/6]	23
6.3.2.2 Vector() [2/6]	24
6.3.2.3 Vector() [3/6]	24
6.3.2.4 Vector() [4/6]	24
6.3.2.5 Vector() [5/6]	25
6.3.2.6 Vector() [6/6]	25
6.3.2.7 ~Vector()	25
6.3.3 Metodų Dokumentacija	25
6.3.3.1 at() [1/2]	25
6.3.3.2 at() [2/2]	26
6.3.3.3 back() [1/2]	26
6.3.3.4 back() [2/2]	27
6.3.3.5 begin() [1/2]	27
6.3.3.6 begin() [2/2]	27
6.3.3.7 capacity()	27
6.3.3.8 cbegin()	28
6.3.3.9 cend()	28
6.3.3.10 clear()	28
6.3.3.11 crbegin()	28
6.3.3.12 crend()	29
6.3.3.13 data()	29

6.3.3.14 empty()	29
6.3.3.15 end() [1/2]	29
6.3.3.16 end() [2/2]	30
6.3.3.17 erase() [1/4]	30
6.3.3.18 erase() [2/4]	30
6.3.3.19 erase() [3/4]	31
6.3.3.20 erase() [4/4]	31
6.3.3.21 front() [1/2]	31
6.3.3.22 front() [2/2]	32
6.3.3.23 insert() [1/3]	32
6.3.3.24 insert() [2/3]	32
6.3.3.25 insert() [3/3]	33
6.3.3.26 operator=() [1/2]	33
6.3.3.27 operator=() [2/2]	34
6.3.3.28 operator[]() [1/2]	34
6.3.3.29 operator[]() [2/2]	34
6.3.3.30 pop_back()	35
6.3.3.31 push_back() [1/2]	35
6.3.3.32 push_back() [2/2]	36
6.3.3.33 rbegin() [1/2]	36
6.3.3.34 rbegin() [2/2]	36
6.3.3.35 rend() [1/2]	36
6.3.3.36 rend() [2/2]	37
6.3.3.37 reserve()	37
6.3.3.38 resize() [1/2]	37
6.3.3.39 resize() [2/2]	38
6.3.3.40 shrink_to_fit()	38
6.3.3.41 size()	38
6.3.3.42 swap()	38
6.4 Zmogus Klasė	39
6.4.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	40
6.4.1.1 Zmogus() [1/4]	40
6.4.1.2 Zmogus() [2/4]	40
6.4.1.3 Zmogus() [3/4]	40
6.4.1.4 Zmogus() [4/4]	41
6.4.1.5 ~Zmogus()	41
6.4.2 Metodų Dokumentacija	41
6.4.2.1 ivest_varda_pavarde()	41
6.4.2.2 nust_pavarde()	42
6.4.2.3 nust_varda()	42
6.4.2.4 pavarde()	42
6.4.2.5 vardas()	42

6.4.3 Atributų Dokumentacija	42
6.4.3.1 pavarde_	42
6.4.3.2 vardas_	42
7 Failo Dokumentacija	43
7.1 src/isvesties_pagalb_fjos.cpp Failo Nuoroda	43
7.1.1 Funkcijos Dokumentacija	43
7.1.1.1 pagal_mediana_did()	43
7.1.1.2 pagal_mediana_maz()	44
7.1.1.3 pagal_pavarde_did()	44
7.1.1.4 pagal_pavarde_maz()	44
7.1.1.5 pagal_varda_did()	44
7.1.1.6 pagal_varda_maz()	44
7.1.1.7 pagal_vidurki_did()	44
7.1.1.8 pagal_vidurki_maz()	44
7.1.1.9 ras_failo_paruosimas()	44
7.2 src/isvesties_pagalb_fjos.h Failo Nuoroda	45
7.2.1 Funkcijos Dokumentacija	45
7.2.1.1 pagal_mediana_did()	45
7.2.1.2 pagal_mediana_maz()	45
7.2.1.3 pagal_pavarde_did()	45
7.2.1.4 pagal_pavarde_maz()	45
7.2.1.5 pagal_varda_did()	46
7.2.1.6 pagal_varda_maz()	46
7.2.1.7 pagal_vidurki_did()	46
7.2.1.8 pagal_vidurki_maz()	46
7.2.1.9 ras_failo_paruosimas()	46
7.2.1.10 rikiavimas()	46
7.2.1.11 stud_rikiavimas()	46
7.3 isvesties_pagalb_fjos.h	47
7.4 src/isvestis.cpp Failo Nuoroda	47
7.4.1 Funkcijos Dokumentacija	48
7.4.1.1 isvestis()	48
7.4.1.2 skirstoma_isvestis()	48
7.4.1.3 spausdinimas()	48
7.4.1.4 studentu_skirstymas()	48
7.4.1.5 visu_stud_duomenu_generavimo_isvestis()	49
7.5 src/isvestis.h Failo Nuoroda	49
7.5.1 Funkcijos Dokumentacija	49
7.5.1.1 isvestis()	49
7.5.1.2 skirstoma_isvestis()	49
7.5.1.3 spausdinimas()	50

7.5.1.4 studentu_skirstymas()	50
7.5.1.5 visu_stud_duomenu_generavimo_investis()	50
7.6 isvestis.h	50
7.7 src/ivesties_pagalb_fjos.cpp Failo Nuoroda	50
7.7.1 Funkcijos Dokumentacija	51
7.7.1.1 ar_sk_ne_tarp_0_ir_10()	51
7.7.1.2 ar_sk_nedidesnis_uz_0()	51
7.7.1.3 ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil()	51
7.7.1.4 ar_sk_nera_1_2_3_4_5_6()	51
7.7.1.5 natur_skaiciaus_investis()	51
7.7.1.6 skait_failo_paruosimas()	52
7.7.1.7 vardo_pavardes_investis()	52
7.8 src/ivesties_pagalb_fjos.h Failo Nuoroda	52
7.8.1 Funkcijos Dokumentacija	52
7.8.1.1 ar_sk_ne_tarp_0_ir_10()	52
7.8.1.2 ar_sk_nedidesnis_uz_0()	52
7.8.1.3 ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil()	52
7.8.1.4 ar_sk_nera_1_2_3_4_5_6()	52
7.8.1.5 natur_skaiciaus_investis()	53
7.8.1.6 skait_failo_paruosimas()	53
7.8.1.7 vardo_pavardes_investis()	53
7.9 ivesties_pagalb_fjos.h	53
7.10 src/ivestis.cpp Failo Nuoroda	53
7.10.1 Funkcijos Dokumentacija	54
7.10.1.1 failo_investis()	54
7.10.1.2 generuota_investis()	54
7.10.1.3 misri_investis()	54
7.10.1.4 rank_investis()	54
7.10.1.5 visu_stud_duomenu_generavimo_investis()	54
7.11 src/ivestis.h Failo Nuoroda	55
7.11.1 Funkcijos Dokumentacija	55
7.11.1.1 failo_investis()	55
7.11.1.2 generuota_investis()	55
7.11.1.3 misri_investis()	55
7.11.1.4 rank_investis()	55
7.11.1.5 visu_stud_duomenu_generavimo_investis()	55
7.12 ivestis.h	56
7.13 src/klaidu_valdymas.cpp Failo Nuoroda	56
7.13.1 Funkcijos Dokumentacija	56
7.13.1.1 ivesties_klaidos_valdymas()	56
7.14 src/klaidu_valdymas.h Failo Nuoroda	56
7.14.1 Funkcijos Dokumentacija	56

7.14.1.1 ivesties_klaidos_valdymas()	56
7.15 klaidu_valdymas.h	57
7.16 src/konstantos_kt_klases.h Failo Nuoroda	57
7.16.1 Apibrėžimų Dokumentacija	57
7.16.1.1 Container	57
7.16.2 Kintamojo Dokumentacija	57
7.16.2.1 MAX_INT	57
7.17 konstantos_kt_klases.h	58
7.18 src/kt_klasu_metodai.cpp Failo Nuoroda	58
7.19 src/pagr.cpp Failo Nuoroda	58
7.19.1 Funkcijos Dokumentacija	59
7.19.1.1 main()	59
7.20 src/studentas.cpp Failo Nuoroda	59
7.20.1 Funkcijos Dokumentacija	59
7.20.1.1 operator<<()	59
7.20.1.2 operator>>()	60
7.21 src/studentas.h Failo Nuoroda	60
7.22 studentas.h	60
7.23 src/vector.h Failo Nuoroda	61
7.23.1 Funkcijos Dokumentacija	62
7.23.1.1 erase()	62
7.23.1.2 erase_if()	63
7.23.1.3 operator!=(())	63
7.23.1.4 operator==(())	64
7.23.1.5 swap()	64
7.24 vector.h	65
7.25 src/zmogus.h Failo Nuoroda	72
7.26 zmogus.h	73
Rodyklė	75

skyrius 1

Direktorių hierarchija

1.1 Direktorijos

src	9
isvesties_pagalb_fjos.cpp	43
isvesties_pagalb_fjos.h	45
isvestis.cpp	47
isvestis.h	49
investies_pagalb_fjos.cpp	50
investies_pagalb_fjos.h	52
investis.cpp	53
investis.h	55
klaidu_valdymas.cpp	56
klaidu_valdymas.h	56
konstantos_kt_klases.h	57
kt_klasiu_metodai.cpp	58
pagr.cpp	58
studentas.cpp	59
studentas.h	60
vector.h	61
zmogus.h	72

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Programos_laikai	11
Vector< T >	20
Zmogus	39
Studentas	12

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Programos_laikai	11
Studentas	12
Vector< T >	20
Zmogus	39

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

src/isvesties_pagalb_fjos.cpp	43
src/isvesties_pagalb_fjos.h	45
src/isvestis.cpp	47
src/isvestis.h	49
src/ivesties_pagalb_fjos.cpp	50
src/ivesties_pagalb_fjos.h	52
src/ivestis.cpp	53
src/ivestis.h	55
src/klaidu_valdymas.cpp	56
src/klaidu_valdymas.h	56
src/konstantos_kt_klases.h	57
src/kt_klasiu_metodai.cpp	58
src/pagr.cpp	58
src/studentas.cpp	59
src/studentas.h	60
src/vector.h	61
src/zmogus.h	72

skyrius 5

Direktorių dokumentacija

5.1 src Direktorijos aprašas

Failai

- failas [isvesties_pagalb_fjos.cpp](#)
- failas [isvesties_pagalb_fjos.h](#)
- failas [isvestis.cpp](#)
- failas [isvestis.h](#)
- failas [ivesties_pagalb_fjos.cpp](#)
- failas [ivesties_pagalb_fjos.h](#)
- failas [ivestis.cpp](#)
- failas [ivestis.h](#)
- failas [klaidu_valdymas.cpp](#)
- failas [klaidu_valdymas.h](#)
- failas [konstantos_kt_klases.h](#)
- failas [kt_klasiu_metodai.cpp](#)
- failas [pagr.cpp](#)
- failas [studentas.cpp](#)
- failas [studentas.h](#)
- failas [vector.h](#)
- failas [zmogus.h](#)

skyrius 6

Klasės Dokumentacija

6.1 Programos_laikai Klasė

```
#include <konstantos_kt_klases.h>
```

Vieši Metodai

- void [spausd_laikus](#) ()

Vieši Atributai

- std::chrono::duration< double > [duomenu_nuskaitymas](#)
- std::chrono::duration< double > [duomenu_rikiavimas](#)
- std::chrono::duration< double > [studentu_skirstymas](#)
- std::chrono::duration< double > [visa_trukme_su_ivestim](#) = std::chrono::milliseconds::zero()

6.1.1 Metodų Dokumentacija

6.1.1.1 spausd_laikus()

```
void Programos_laikai::spausd_laikus ()
```

6.1.2 Atributų Dokumentacija

6.1.2.1 duomenu_nuskaitymas

```
std::chrono::duration<double> Programos_laikai::duomenu_nuskaitymas
```

6.1.2.2 duomenu_rikiavimas

```
std::chrono::duration<double> Programos_laikai::duomenu_rikiavimas
```


- float `rezas_med` () const
Grąžina galutinį rezultatą pagal medianą.
- size_t `pazymiu_sk` ()
Grąžina esamą pažymių skaičių.
- bool `invest_varda_pavarde` (bool ar_investis_atsaukiama=false) override
Leidžia vartotojui įvesti vardą ir pavardę rankiniu būdu. Įgyvendina tėvinės žmogaus klasės grynąją virtualiąją funkciją.
- void `pridet_pazymi` (int paz)
Prideda pažymį (nuo 0 iki 10) į sąrašą.
- void `nust_egzo_reza` (int rez)
Nustato egzamino rezultatą.
- void `invest_egzo_reza` ()
Leidžia vartotojui įvesti egzamino rezultatą rankiniu būdu.
- void `apsk_vid` ()
Apskaičiuoja galutinį rezultatą naudojant aritmetinį vidurkį.
- void `apsk_med` ()
Apskaičiuoja galutinį rezultatą naudojant medianą.
- void `uzpildyt_pazymius_iki_min` (int min_pazymiu_sk)
Užpildo pažymių sąrašą nuliais iki nurodyto kiekio.

Vieši Metodai inherited from `Zmogus`

- `Zmogus` ()
Konstruktorius be parametrų.
- `Zmogus` (std::string vardas, std::string pavarde)
Konstruktorius su vardu ir pavarde.
- `Zmogus` (const `Zmogus` &kitas)
Kopijavimo konstruktorius.
- `Zmogus` (`Zmogus` &&kitas)
Perkėlimo konstruktorius.
- `~Zmogus` ()
Destruktorius.
- std::string `vardas` () const
Grąžina žmogaus vardą.
- std::string `pavarde` () const
Grąžina žmogaus pavardę.
- void `nust_varda` (std::string vardas)
Nustato žmogaus vardą.
- void `nust_pavarde` (std::string pavarde)
Nustato žmogaus pavardę.

Statiniai Vieši Metodai

- static std::string `galut` ()
Grąžina esamą galutinio skaičiavimo tipą.
- static void `nust_galutinio_tipa` (std::string v_m)
Nustato, koks galutinio rezultato skaičiavimo tipas naudosimas išvestyje.

Draugai

- `std::ostream & operator<<` (`std::ostream &os, const Studentas &stud`)
Išvesties operatorius studento duomenims spausdinti.
- `std::istream & operator>>` (`std::istream &is, Studentas &stud`)
Įvesties operatorius studento duomenims nuskaityti.

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė.

6.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

6.2.1.1 Studentas() [1/4]

```
Studentas::Studentas () [inline]
```

Konstruktorius be parametrų.

- Sukuria tuščią studento objektą su nulinėmis reikšmėmis.

6.2.1.2 Studentas() [2/4]

```
Studentas::Studentas (
    std::string vardas,
    std::string pavarde) [inline]
```

Konstruktorius su vardu ir pavarde.

- Sukuria studento objektą su vardu ir pavarde, o likusios reikšmės — nulinės.

Parametrai

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavardė.

6.2.1.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & kitas) [inline]
```

Kopijavimo konstruktorius.

- Sukuria naują objektą nukopijuodamas visus duomenis iš kito studento.

Parametrai

<i>kitas</i>	Nuoroda objekto, iš kurio kopijuojami duomenys.
--------------	---

6.2.1.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && kitas) [inline]
```

Perkėlimo konstruktorius.

- Perkelia resursus iš kito objekto į naują, palikdamas senąjį tuščią.

Parametrai

<i>kitas</i>	"rvalue" nuoroda į studento objektą.
--------------	--------------------------------------

6.2.1.5 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Destruktorius.

- Išvalo pažymių konteinerį.

6.2.2 Metodų Dokumentacija

6.2.2.1 apsk_med()

```
void Studentas::apsk_med ()
```

Apskaičiuoja galutinį rezultatą naudojant medianą.

6.2.2.2 apsk_vid()

```
void Studentas::apsk_vid ()
```

Apskaičiuoja galutinį rezultatą naudojant aritmetinį vidurkį.

6.2.2.3 Container()

```
Studentas::Container (
    int ) const [inline]
```

Grąžina pažymių konteinerį.

6.2.2.4 egzo_rezas()

```
int Studentas::egzo_rezas () const [inline]
```

Grąžina egzamino rezultatą.

6.2.2.5 galut()

```
std::string Studentas::galut () [inline], [static]
```

Grąžina esamą galutinio skaičiavimo tipą.

6.2.2.6 iverst_egzo_reza()

```
void Studentas::iverst_egzo_reza ()
```

Leidžia vartotojui įvesti egzamino rezultatą rankiniu būdu.

6.2.2.7 iverst_varda_pavarde()

```
bool Studentas::iverst_varda_pavarde (
    bool ar_iverstis_atsaukiama = false) [override], [virtual]
```

Leidžia vartotojui įvesti vardą ir pavardę rankiniu būdu. Įgyvendina tėvinės žmogaus klasės grynąją virtualiąją funkciją.

Parametrai

<i>ar_iverstis_atsaukiama</i>	Ar leidžiama nutraukti studento duomenų įvedimą įvedus "x". Numatytoji reikšmė:↵ false.
-------------------------------	--

Gražina

true Jei įvesta sėkmingai.

false Jei įvedimas atšauktas.

Realizuoja [Zmogus](#).

6.2.2.8 nust_egzo_reza()

```
void Studentas::nust_egzo_reza (
    int rez)
```

Nustato egzamino rezultatą.

Parametrai

<i>rez</i>	Egzamino rezultatas.
------------	----------------------

6.2.2.9 nust_galutinio_tipa()

```
void Studentas::nust_galutinio_tipa (
    std::string v_m) [static]
```

Nustato, koks galutinio rezultato skaičiavimo tipas naudosimas išvestyje.

Parametrai

$v \leftrightarrow$ <i>_m</i>	"v": vidurkis, "m": mediana
----------------------------------	-----------------------------

6.2.2.10 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>kitas</i>	Nuoroda objekto, kurio duomenys bus nukopijuoti ir priskirti.
--------------	---

Gražina

Studentas& nuoroda į atnaujintą objektą.

6.2.2.11 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas)
```

Perkėlimo priskyrimo operatorius.

- Perkelia resursus iš kito studento objekto į šį. Šaltinis (objektas kitas) tampa galimos, tačiau neapibrėžtos būsenos.

- Parametrai

<i>kitas</i>	"rvalue" nuoroda į perkelsimą studento objektą.
--------------	---

Gražina

[Studentas](#)& nuoroda į šį patį objektą (*this).

6.2.2.12 pazymiu_sk()

```
size_t Studentas::pazymiu_sk () [inline]
```

Gražina esamą pažymių skaičių.

6.2.2.13 prideti_pazymi()

```
void Studentas::prideti_pazymi (
    int paz)
```

Prideda pažymį (nuo 0 iki 10) į sąrašą.

Parametrai

<i>paz</i>	Pridedamas pažymys.
------------	---------------------

6.2.2.14 rezas_med()

```
float Studentas::rezas_med () const [inline]
```

Gražina galutinį rezultatą pagal medianą.

6.2.2.15 rezas_vid()

```
float Studentas::rezas_vid () const [inline]
```

Gražina galutinį rezultatą pagal vidurkį.

6.2.2.16 uždaryti_pazymius_iki_min()

```
void Studentas::uždaryti_pazymius_iki_min (
    int min_pazymiu_sk)
```

Uždarys pažymių sąrašą nuliais iki nurodyto kiekio.

Parametrai

<i>min_pazymiu_sk</i>	Mažiausias privalomas turėti pažymių skaičius.
-----------------------	--

6.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

6.2.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & stud) [friend]
```

Išvesties operatorius studento duomenims spausdinti.

Parametrai

<i>os</i>	Nuoroda į išvesties srautą.
<i>stud</i>	Nuoroda į išvesimą studentą.

Gražina

std::ostream& nuoroda į išvesties srautą (tinkamam operatoriaus veikimui).

6.2.3.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & stud) [friend]
```

Išvesties operatorius studento duomenims nuskaityti.

Parametrai

<i>is</i>	Nuoroda į įvesties srautą.
<i>stud</i>	Nuoroda į studento objektą, į kurį bus nuskaityti duomenys.

Gražina

std::istream& Gražinama nuoroda į įvesties srautą (tinkamam operatoriaus veikimui).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [src/studentas.h](#)
- [src/studentas.cpp](#)

6.3 Vector< T > Klasė Šablonas

```
#include <vector.h>
```

Vieši Tipai

- using `value_type` = T
- using `size_type` = std::size_t
- using `difference_type` = std::ptrdiff_t
- using `reference` = value_type &
- using `const_reference` = const value_type &
- using `pointer` = value_type *
- using `const_pointer` = const value_type *
- using `iterator` = pointer
- using `const_iterator` = const_pointer
- using `reverse_iterator` = std::reverse_iterator<iterator>
- using `const_reverse_iterator` = std::reverse_iterator<const_iterator>

Vieši Metodai

- `Vector` ()
Numatytasis konstruktorius. Sukuria tuščią vektorį.
- `Vector` (size_type n)
Sukuria nurodyto dydžio vektorių, užpildytą numatytosiomis reikšmėmis.
- `Vector` (size_type n, const T &x)
Sukuria vektorių ir užpildo jį nurodyto objekto kopijomis.
- `Vector` (std::initializer_list< T > list)
Sukuria vektorių iš inicializavimo sąrašo (initializer_list).
- `Vector` (const Vector &other)
Kopijavimo konstruktorius. Sukuria naują vektorių kaip kito vektoriaus kopiją.
- `Vector` (Vector &&other) noexcept
Perkėlimo (move) konstruktorius. Perima kito vektoriaus resursus be kopijavimo.
- `~Vector` ()
Destruktorius. Sunaikina visus elementus ir atlaisvina atmintį.
- `Vector & operator=` (const Vector &other)
Kopijavimo priskyrimo operatorius. Pakeičia esamo vektoriaus turinį kito vektoriaus kopija.
- `Vector & operator=` (Vector &&other) noexcept
Perkėlimo priskyrimo operatorius. Pakeičia esamo vektoriaus turinį kito vektoriaus resursais.
- `reference at` (size_type i)
Saugiai pasiekia elementą nurodytu indeksu atlikdamas ribų patikrinimą.
- `const_reference at` (size_type i) const
Saugiai pasiekia elementą nurodytu indeksu atlikdamas ribų patikrinimą.
- `reference operator[]` (size_type i)
Pasiekia elementą nurodytu indeksu be ribų patikrinimo.
- `const_reference operator[]` (size_type i) const
Pasiekia elementą nurodytu indeksu be ribų patikrinimo (tik skaityti).
- `reference front` ()
Grąžina nuorodą į pirmąjį vektoriaus elementą.
- `const_reference front` () const
Grąžina nuorodą į pirmąjį vektoriaus elementą (tik skaityti).

- `reference back ()`
Grąžina nuorodą į paskutinį vektoriaus elementą.
- `const_reference back () const`
Grąžina nuorodą į paskutinį vektoriaus elementą (tik skaityti).
- `T * data ()`
Grąžina tiesioginę rodyklę į vidinį elementų masyvą.
- `iterator begin ()`
Grąžina iteratorių į pirmąjį vektoriaus elementą.
- `const_iterator begin () const`
Grąžina konstantinį iteratorių į pirmąjį vektoriaus elementą.
- `const_iterator cbegin () const`
Grąžina konstantinį iteratorių į pirmąjį vektoriaus elementą.
- `iterator end ()`
Grąžina iteratorių į vietą už paskutinio vektoriaus elemento.
- `const_iterator end () const`
Grąžina konstantinį iteratorių į vietą už paskutinio vektoriaus elemento.
- `const_iterator cend () const`
Grąžina konstantinį iteratorių į vietą už paskutinio vektoriaus elemento.
- `reverse_iterator rbegin ()`
Grąžina atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.
- `const_reverse_iterator rbegin () const`
Grąžina konstantinį atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.
- `const_reverse_iterator crbegin () const`
Grąžina konstantinį atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.
- `reverse_iterator rend ()`
Grąžina atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.
- `const_reverse_iterator rend () const`
Grąžina konstantinį atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.
- `const_reverse_iterator crend () const`
Grąžina konstantinį atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.
- `bool empty () const`
Patikrina, ar vektorius yra tuščias.
- `size_type size () const`
Grąžina vektoriuje esančių elementų skaičių.
- `void reserve (size_type new_cap)`
Padidina vektoriaus vidinę talpą (išskirtą atminti), kad joje tilptų nurodytas elementų kiekis.
- `size_type capacity () const`
Grąžina vektoriaus talpą.
- `void shrink_to_fit ()`
*Sumažina išskirtos atminties talpą (*capacity*), kad ji sutaptų su esamu elementų kiekiu (*size*).*
- `void clear ()`
Ištrina visus elementus iš vektoriaus.
- `void push_back (const T &value)`
Prideda elementą (kopijuojant) į vektoriaus pabaigą.
- `void push_back (T &&value)`
Prideda elementą (perkėliant) į vektoriaus pabaigą.
- `void pop_back ()`
Ištrina paskutinį vektoriaus elementą.
- `iterator insert (const_iterator pos, const T &value)`
Įterpia elemento kopiją prieš nurodytą poziciją.
- `iterator insert (const_iterator pos, T &&value)`

- *Įterpia elementą (perkeliant) prieš nurodytą poziciją.*
- `iterator insert (const_iterator pos, size_type count, const T &value)`
Įterpia nurodytą kiekį elemento kopijų prieš nurodytą poziciją.
- `iterator erase (iterator pos)`
Pašalina elementą nurodytoje pozicijoje.
- `iterator erase (const_iterator pos)`
Pašalina elementą nurodytoje pozicijoje.
- `iterator erase (iterator first, iterator last)`
Pašalina elementų rėžį [first, last).
- `iterator erase (const_iterator first, const_iterator last)`
Pašalina elementų rėžį [first, last).
- `void resize (size_type count)`
Pakeičia vektoriaus dydį iki nurodyto elementų skaičiaus.
- `void resize (size_type count, const T &value)`
Pakeičia vektoriaus dydį iki nurodyto elementų skaičiaus.
- `void swap (Vector &other) noexcept`
Sukeičia šio vektoriaus turinį (duomenis, dydį ir talpą) su kitu vektoriu.

6.3.1 Tipo Aprašymo Dokumentacija

6.3.1.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const_pointer
```

6.3.1.2 const_pointer

```
template<typename T>
using Vector< T >::const_pointer = const value_type *
```

6.3.1.3 const_reference

```
template<typename T>
using Vector< T >::const_reference = const value_type &
```

6.3.1.4 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

6.3.1.5 difference_type

```
template<typename T>
using Vector< T >::difference_type = std::ptrdiff_t
```

6.3.1.6 iterator

```
template<typename T>
using Vector< T >::iterator = pointer
```

6.3.1.7 pointer

```
template<typename T>
using Vector< T >::pointer = value_type *
```

6.3.1.8 reference

```
template<typename T>
using Vector< T >::reference = value_type &
```

6.3.1.9 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

6.3.1.10 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

6.3.1.11 value_type

```
template<typename T>
using Vector< T >::value_type = T
```

6.3.2 Konstruktorius ir Destruktorius Dokumentacija

6.3.2.1 Vector() [1/6]

```
template<typename T>
Vector< T >::Vector () [inline]
```

Numatytasis konstruktorius. Sukuria tuščią vektorių.

Pradinė talpa ir dydis yra lygi nuliui, atmintis nėra išskiriama.

6.3.2.2 Vector() [2/6]

```
template<typename T>
Vector< T >::Vector (
    size_type n) [inline]
```

Sukuria nurodyto dydžio vektorių, užpildytą numatytosiomis reikšmėmis.

Atmintis rezervuojama naudojant operator `new`, o objektai sukonstruojami naudojant `std::uninitialized_construct_n`. Tai užtikrina, kad primityvūs tipai (pvz., `int`) bus inicializuoti nuliais, o ne atminties "šiukšlėmis".

Parametrai

<i>n</i>	Pradinis vektoriaus dydis ir talpa.
----------	-------------------------------------

6.3.2.3 Vector() [3/6]

```
template<typename T>
Vector< T >::Vector (
    size_type n,
    const T & x) [inline]
```

Sukuria vektorių ir užpildo jį nurodyto objekto kopijomis.

Parametrai

<i>n</i>	Pradinis vektoriaus dydis ir talpa.
<i>x</i>	Reikšmė, kuria bus užpildyti visi elementai.

6.3.2.4 Vector() [4/6]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > list) [inline]
```

Sukuria vektorių iš inicializavimo sąrašo (`initializer_list`).

Parametrai

<i>list</i>	Sąrašas elementų, kurie bus nukopijuoti į naująjį vektorių.
-------------	---

6.3.2.5 Vector() [5/6]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

Kopijavimo konstruktorius. Sukuria naują vektorių kaip kito vektoriaus kopiją.

Parametrai

<i>other</i>	Vektorius, iš kurio kopijuojami elementai.
--------------	--

6.3.2.6 Vector() [6/6]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

Perkėlimo (move) konstruktorius. Perima kito vektoriaus resursus be kopijavimo.

Po šios operacijos pradinis vektorius *other* tampa tuščias (galimoje, bet neapibrėžtoje būsenoje).

Parametrai

<i>other</i>	Vektorius, kurio resursai bus pasisavinti.
--------------	--

6.3.2.7 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

Destruktorius. Sunaikina visus elementus ir atlaisvina atmintį.

Sunaikinimas vyksta dviem etapais: pirmiausia `std::destroy` iškviečia visų objektų destruktoriaus, tada `operator delete` atlaisvina pačią atmintį.

6.3.3 Metodų Dokumentacija

6.3.3.1 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
    size_type i) [inline]
```

Saugiai pasiekia elementą nurodytu indeksu atlikdamas ribų patikrinimą.

Jei nurodytas indeksas yra už vektoriaus ribų, funkcija meta `std::out_of_range` išimtį.

Parametrai

<i>i</i>	Norimo pasiekti elemento indeksas.
----------	------------------------------------

Gražina

Nuoroda (reference) į nurodytą elementą.

Išimtys

<i>std::out_of_range</i>	Jei indeksas yra didesnis arba lygus vektoriaus dydžiui ($i \geq \text{size_}$).
--------------------------	---

6.3.3.2 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
    size_type i) const [inline]
```

Saugiai pasiekia elementą nurodytu indeksu atlikdamas ribų patikrinimą.

Jei nurodytas indeksas yra už vektoriaus ribų, funkcija meta `std::out_of_range` išimtį.

Parametrai

<i>i</i>	Norimo pasiekti elemento indeksas.
----------	------------------------------------

Gražina

Konstantinė nuoroda (reference) į nurodytą elementą.

Išimtys

<i>std::out_of_range</i>	Jei indeksas yra didesnis arba lygus vektoriaus dydžiui ($i \geq \text{size_}$).
--------------------------	---

6.3.3.3 back() [1/2]

```
template<typename T>
reference Vector< T >::back () [inline]
```

Gražina nuorodą į paskutinį vektoriaus elementą.

Gražina

Nuoroda į paskutinį elementą.

Išpėjimas

Iškvietus šį metodą tuščiam vektoriui, elgsena neapibrėžta.

6.3.3.4 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const [inline]
```

Grąžina nuorodą į paskutinį vektoriaus elementą (tik skaityti).

Gražina

Konstantinė nuoroda į paskutinį elementą.

Ispėjimas

Iškvietus šį metodą tuščiam vektoriui, elgsena neapibrėžta.

6.3.3.5 begin() [1/2]

```
template<typename T>
iterator Vector< T >::begin () [inline]
```

Grąžina iteratorių į pirmąjį vektoriaus elementą.

Gražina

Iteratorius, rodantis į pradžią.

6.3.3.6 begin() [2/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline]
```

Grąžina konstantinį iteratorių į pirmąjį vektoriaus elementą.

Gražina

Konstantinis iteratorius, rodantis į pradžią.

6.3.3.7 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const [inline]
```

Grąžina vektoriaus talpą.

Vektoriaus talpa — elementų skaičius, kurį vektorius gali sutalpinti be papildomo atminties išskyrimo. Šis skaičius visada didesnis arba lygus `size()`.

Gražina

Esama vektoriaus atminties talpa.

6.3.3.8 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline]
```

Grąžina konstantinį iteratorių į pirmąjį vektoriaus elementą.

Naudojamas, kai norima užtikrinti, kad per iteratorių duomenys nebus keičiami, net jei pats vektorius nėra konstantinis.

Gražina

Konstantinis iteratorius, rodantis į pradžią.

6.3.3.9 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline]
```

Grąžina konstantinį iteratorių į vietą už paskutinio vektoriaus elemento.

Gražina

Konstantinis iteratorius, rodantis į pabaigą (už paskutinio elemento).

6.3.3.10 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

Ištrina visus elementus iš vektoriaus.

Visiems vektoriuje esantiems objektams saugiai iškviečiami jų destruktoriai, tačiau vidinė atmintis (talpa /↔ `capacity`) nėra atlaisvinama. Po šios operacijos `size()` tampa 0.

6.3.3.11 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline]
```

Grąžina konstantinį atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.

Gražina

Konstantinis atvirkštinis iteratorius, rodantis į paskutinį elementą.

6.3.3.12 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline]
```

Gražina konstantinį atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.

Gražina

Konstantinis atvirkštinis iteratorius, rodantis į vietą prieš pirmąjį elementą.

6.3.3.13 data()

```
template<typename T>
T * Vector< T >::data () [inline]
```

Gražina tiesioginę rodyklę į vidinį elementų masyvą.

Gražina

Rodyklė į pirmąjį masyvo elementą arba `nullptr`, jei atmintis neišskirta.

6.3.3.14 empty()

```
template<typename T>
bool Vector< T >::empty () const [inline]
```

Patikrina, ar vektorius yra tuščias.

Tikrinimas atliekamas lyginant pradžios ir pabaigos iteratorius.

Gražina

`true`, jei `begin()` == `end()` (t. y. vektoriuje nėra elementų: `size()` == 0), kitu atveju `false`.

6.3.3.15 end() [1/2]

```
template<typename T>
iterator Vector< T >::end () [inline]
```

Gražina iteratorių į vietą už paskutinio vektoriaus elemento.

Gražina

Iteratorius, rodantis į pabaigą (už paskutinio elemento).

6.3.3.16 end() [2/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline]
```

Gražina konstantinį iteratorių į vietą už paskutinio vektoriaus elemento.

Gražina

Konstantinis iteratorius, rodantis į pabaigą (už paskutinio elemento).

6.3.3.17 erase() [1/4]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Pašalina elementų rėžį [first, last).

Elementai po nurodyto rėžio perkeliama į atsilaisvinusią vietą.

Parametrai

<i>first</i>	Konstantinis iteratorius į rėžio pradžią (įskaitant).
<i>last</i>	Konstantinis iteratorius į rėžio pabaigą (neįskaitant).

Gražina

Iteratorius į elementą, sekantį po paskutinio pašalinto elemento.

6.3.3.18 erase() [2/4]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

Pašalina elementą nurodytoje pozicijoje.

Visi elementai po nurodytos pozicijos perkeliama per vieną vietą atgal naudojant perkėlimo (move) priskyrimą. Visi iteratoriai ir nuorodos nuo ištrynimo vietos iki vektoriaus galo tampa negaliojantys.

Parametrai

<i>pos</i>	Konstantinis iteratorius į elementą, kurį reikia ištrinti.
------------	--

Gražina

Iteratorius į elementą, kuris sekė po ištrintojo (arba `end()`).

6.3.3.19 erase() [3/4]

```
template<typename T>
iterator Vector< T >::erase (
    iterator first,
    iterator last) [inline]
```

Pašalina elementų rėžį [first, last).

Elementai po nurodyto rėžio perkeliama į atsilaisvinusią vietą.

Parametrai

<i>first</i>	Iteratorius į rėžio pradžią (įskaitant).
<i>last</i>	Iteratorius į rėžio pabaigą (neįskaitant).

Gražina

Iteratorius į elementą, sekantį po paskutinio pašalinto elemento.

6.3.3.20 erase() [4/4]

```
template<typename T>
iterator Vector< T >::erase (
    iterator pos) [inline]
```

Pašalina elementą nurodytoje pozicijoje.

Visi elementai po nurodytos pozicijos perkeliama per vieną vietą atgal naudojant perkėlimo (move) priskyrimą. Visi iteratoriai ir nuorodos nuo ištrynimo vietos iki vektoriaus galo tampa negaliojantys.

Parametrai

<i>pos</i>	Iteratorius į elementą, kurį reikia ištrinti.
------------	---

Gražina

Iteratorius į elementą, kuris sekė po ištrintojo (arba `end()`).

6.3.3.21 front() [1/2]

```
template<typename T>
reference Vector< T >::front () [inline]
```

Gražina nuorodą į pirmąjį vektoriaus elementą.

Gražina

Nuoroda į pirmąjį elementą.

Išpėjimas

Iškvietus šį metodą tuščiam vektoriui, elgsena neapibrėžta.

6.3.3.22 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const [inline]
```

Gražina nuorodą į pirmąjį vektoriaus elementą (tik skaityti).

Gražina

Konstantinė nuoroda į pirmąjį elementą.

Išėjimas

Iškvietus šį metodą tuščiam vektoriui, elgsena neapibrėžta.

6.3.3.23 insert() [1/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value) [inline]
```

Įterpia elemento kopiją prieš nurodytą poziciją.

Visi elementai nuo iteratoriaus `pos` iki pabaigos yra pastumiami per vieną poziciją tolyn. Jei įvyksta atminties perskirstymas, visi iteratoriai ir nuorodos tampa nebegaliojantys. Kitu atveju tampa nebegaliojantys tik iteratoriai nuo `pos` iki pabaigos.

Parametrai

<i>pos</i>	Iteratorius, prieš kurį bus įterpiamas elementas.
<i>value</i>	Elementas, kuris bus nukopijuotas į vektorių.

Gražina

Iteratorius, rodantis į naujai įterptą elementą.

6.3.3.24 insert() [2/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value) [inline]
```

Įterpia nurodytą kiekį elemento kopijų prieš nurodytą poziciją.

Jei įvyksta atminties perskirstymas, visi iteratoriai ir nuorodos tampa nebegaliojantys. Kitu atveju tampa nebegaliojantys tik iteratoriai nuo `pos` iki pabaigos.

Parametrai

<i>pos</i>	Iteratorius, prieš kurį bus įterpiami nauji elementai.
<i>count</i>	Įterpiamų elementų skaičius. Jei jis 0, funkcija nieko nedaro.
<i>value</i>	Elemento reikšmė, kurios kopijos bus įterptos.

Gražina

Iteratorius, nurodantis į pirmąjį naujai įterptą elementą (arba `pos`, jei `count == 0`).

6.3.3.25 insert() [3/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    T && value) [inline]
```

Įterpia elementą (perkeliant) prieš nurodytą poziciją.

Visi elementai nuo iteratoriaus `pos` iki pabaigos yra pastumiami per vieną poziciją tolyn. Jei įvyksta atminties perskirstymas, visi iteratoriai ir nuorodos tampa nebegaliojantys. Kitu atveju tampa nebegaliojantys tik iteratoriai nuo `pos` iki pabaigos.

Parametrai

<i>pos</i>	Iteratorius, prieš kurį bus įterpiamas elementas.
<i>value</i>	Elementas, kuris bus perkeltas į vektorių. Po operacijos senasis perduotasis elementas liks galimoje, bet neapibrėžtoje būsenoje.

Gražina

Iteratorius, rodantis į naujai įterptą elementą.

6.3.3.26 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

Kopijavimo priskyrimo operatorius. Pakeičia esamo vektoriaus turinį kito vektoriaus kopija.

Parametrai

<i>other</i>	Vektorius, kurio elementai bus nukopijuoti.
--------------	---

Gražina

Nuoroda į šį (modifikuotą) vektorių.

6.3.3.27 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius. Pakeičia esamo vektoriaus turinį kito vektoriaus resursais.

Senieji elementai yra sunaikinami, o senoji atmintis atlaisvinama.

Parametrai

<i>other</i>	Vektorius, kurio resursai bus pasisavinti. Po operacijos jis tampa tuščias.
--------------	---

Gražina

Nuoroda į šį vektorių.

6.3.3.28 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
    size_type i) [inline]
```

Pasiekia elementą nurodytu indeksu be ribų patikrinimo.

Šis metodas yra greitesnis nei `at()`, nes netikrina, ar indeksas yra tinkamas. Mėginimas nurodyti `i >= size()` lems neapibrėžtą programos elgseną.

Parametrai

<i>i</i>	Norimo pasiekti elemento indeksas.
----------	------------------------------------

Gražina

Nuoroda (reference) į nurodytą elementą.

6.3.3.29 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
    size_type i) const [inline]
```

Pasiekia elementą nurodytu indeksu be ribų patikrinimo (tik skaityti).

Šis metodas yra greitesnis nei `at()`, nes netikrina, ar indeksas tinkamas.

Parametrai

<i>i</i>	Norimo pasiekti elemento indeksas.
----------	------------------------------------

Gražina

Konstantinė nuoroda (reference) į nurodytą elementą.

Išpėjimas

Jei indeksas yra už vektoriaus ribų, elgsena neapibrėžta.

6.3.3.30 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

Ištrina paskutinį vektoriaus elementą.

Sunaikina paskutinį objektą iškviėsdamas jo destruktorių, bet neatlaisvina vidinės atminties (`capacity_` nepakinta). Jei vektorius jau yra tuščias, funkcija nieko nedaro. Po operacijos iteratoriai, rodyklės ir nuorodos į pašalintąjį elementą ir pabaigos iteratorių (`end()`) tampa nebegaliojantys.

6.3.3.31 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

Prideda elementą (kopijuojant) į vektoriaus pabaigą.

Jei dabartinis dydis (`size_`) pasiekia talpą (`capacity_`), prieš pridedant elementą automatiškai išskiriama nauja atmintis (paprastai dvigubai didesnė). Tokiu atveju visi esami iteratoriai, rodyklės į vektoriaus elementus tampa nebegaliojantys.

Parametrai

<i>value</i>	Pridedamo elemento reikšmė (nukopijuosima).
--------------	---

6.3.3.32 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

Prideda elementą (perkėliant) į vektoriaus pabaigą.

Jei dabartinis dydis (`size_`) pasiekia talpą (`capacity_`), prieš pridedant elementą automatiškai išskiriama nauja atmintis (paprastai dvigubai didesnė). Tokiu atveju visi esami iteratoriai, rodyklės ir nuorodos į vektoriaus elementus tampa nebegaliojantys.

Parametrai

<i>value</i>	Pridedamo elemento reikšmė (perkelsima). Po perkėlimo senasis elementas liks galimoje, bet neapibrėžtoje būsenoje.
--------------	--

6.3.3.33 rbegin() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline]
```

Gražina atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.

Gražina

Atvirkštinis iteratorius, rodantis į paskutinį elementą.

6.3.3.34 rbegin() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline]
```

Gražina konstantinį atvirkštinį iteratorių į pirmąjį atvirkštinės sekos elementą.

Gražina

Konstantinis atvirkštinis iteratorius, rodantis į paskutinį elementą.

6.3.3.35 rend() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline]
```

Gražina atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.

Gražina

Atvirkštinis iteratorius, rodantis į vietą prieš pirmąjį elementą.

6.3.3.36 `rend()` [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline]
```

Gražina konstantinį atvirkštinį iteratorių į vietą už paskutinio atvirkštinės sekos elemento.

Gražina

Konstantinis atvirkštinis iteratorius, rodantis į vietą prieš pirmąjį elementą.

6.3.3.37 `reserve()`

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap) [inline]
```

Padidina vektoriaus vidinę talpą (išskirtą atmintį), kad joje tilptų nurodytas elementų kiekis.

Jei nurodyta talpa (`new_cap`) yra mažesnė arba lygi dabartinei talpai (`capacity_`), funkcija nieko nedaro. Jei reikia didinti talpą, išskiriama nauja atmintis, senieji objektai perkeliama į naują vietą, o senoji atmintis atlaisvinama.

Parametrai

<code>new_cap</code>	Norimas rezervuoti atminties dydis (elementų skaičiumi).
----------------------	--

6.3.3.38 `resize()` [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count) [inline]
```

Pakeičia vektoriaus dydį iki nurodyto elementų skaičiaus.

Jei naujas dydis mažesnis už esamą, elementai vektoriaus gale sunaikinami. Jei naujas dydis didesnis, vektorius papildomas reikšmėmis pagal "value-initialization" (pvz., skaičiai tampa 0). Jei reikia, padidinama talpa.

Parametrai

<code>count</code>	Naujas norimas vektoriaus dydis.
--------------------	----------------------------------

6.3.3.39 `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const T & value) [inline]
```

Pakeičia vektoriaus dydį iki nurodyto elementų skaičiaus.

Jei naujas dydis mažesnis už esamą, elementai vektoriaus gale sunaikinami. Jei naujas dydis didesnis, vektorius papildomas nurodytomis reikšmėmis `value`. Jei reikia, padidinama talpa.

Parametrai

<i>count</i>	Naujas norimas vektoriaus dydis.
<i>value</i>	Reikšmė, kuria bus užpildyti naujai sukurti elementai.

6.3.3.40 `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

Sumažina išskirtos atminties talpą (`capacity`), kad ji sutaptų su esamu elementų kiekiu (`size`).

Funkcija naudinga siekiant optimizuoti atminties naudojimą.

6.3.3.41 `size()`

```
template<typename T>
size_type Vector< T >::size () const [inline]
```

Grąžina vektoriuje esančių elementų skaičių.

Gražina

Elementų kiekis vektoriuje.

6.3.3.42 `swap()`

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

Sukeičia šio vektoriaus turinį (duomenis, dydį ir talpą) su kitu vektoriu.

Operacijoje tik sukeičiamos vidinės rodyklės ir kintamieji, neatliekant elementų kopijavimo ar perkėlimo.

Parametrai

<i>other</i>	Kitas vektorius, su kuriuo bus atliekamas sukeitimas.
--------------	---

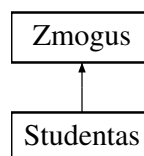
Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/vector.h](#)

6.4 Zmogus Klasė

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:

**Vieši Metodai**

- [Zmogus \(\)](#)
Konstruktorius be parametru.
- [Zmogus \(std::string vardas, std::string pavarde\)](#)
Konstruktorius su vardu ir pavarde.
- [Zmogus \(const Zmogus &kitas\)](#)
Kopijavimo konstruktorius.
- [Zmogus \(Zmogus &&kitas\)](#)
Perkėlimo konstruktorius.
- [~Zmogus \(\)](#)
Destruktorius.
- [std::string vardas \(\) const](#)
Grąžina žmogaus vardą.
- [std::string pavarde \(\) const](#)
Grąžina žmogaus pavardę.
- [void nust_varda \(std::string vardas\)](#)
Nustato žmogaus vardą.
- [void nust_pavarde \(std::string pavarde\)](#)
Nustato žmogaus pavardę.
- [virtual bool iverst_varda_pavarde \(bool ar_iverstis_atsaukiama=false\)=0](#)
Grynoji virtualioji funkcija, leisianti vartotojui įvesti išvestinių klasių objektų vardą ir pavardę rankiniu būdu.

Apsaugoti Atributai

- [std::string vardas_](#)
Žmogaus vardas.
- [std::string pavarde_](#)
Žmogaus pavardė.

6.4.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

6.4.1.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

Konstruktorius be parametrų.

- Sukuria tuščią žmogaus objektą su nulinėmis reikšmėmis.

6.4.1.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    std::string vardas,  
    std::string pavarde) [inline]
```

Konstruktorius su vardu ir pavarde.

- Sukuria žmogaus objektą su vardu ir pavarde.

Parametrai

<i>vardas</i>	Žmogaus vardas.
<i>pavarde</i>	Žmogaus pavardė.

6.4.1.3 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & kitas) [inline]
```

Kopijavimo konstruktorius.

- Sukuria naują objektą nukopijuodamas visus duomenis iš kito studento.

Parametrai

<i>kitas</i>	Nuoroda objekto, iš kurio kopijuojami duomenys.
--------------	---

6.4.1.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && kitas) [inline]
```

Perkėlimo konstruktorius.

- Perkelia resursus iš kito objekto į naują, palikdamas senąjį tuščią.

Parametrai

<i>kitas</i>	"rvalue" nuoroda į žmogaus objektą.
--------------	-------------------------------------

6.4.1.5 ~Zmogus()

```
Zmogus::~~Zmogus () [inline]
```

Destruktorius.

6.4.2 Metodų Dokumentacija

6.4.2.1 iverst_varda_pavarde()

```
virtual bool Zmogus::iverst_varda_pavarde (
    bool ar_iverstis_atsaukiama = false) [pure virtual]
```

Grynoji virtualioji funkcija, leisianti vartotojui įvesti išvestinių klasių objektų vardą ir pavardę rankiniu būdu.

Parametrai

<i>ar_iverstis_atsaukiama</i>	Ar leidžiama nutraukti studento duomenų įvedimą įvedus "x". Numatytoji reikšmė:↵ false.
-------------------------------	--

Gražina

true Jei įvesta sėkmingai.

false Jei įvedimas atšauktas.

Realizuota [Studentas](#).

6.4.2.2 nust_pavarde()

```
void Zmogus::nust_pavarde (
    std::string pavarde) [inline]
```

Nustato žmogaus pavardę.

Parametrai

<i>vardas</i>	Žmogaus pavardė.
-------------------------------	------------------

6.4.2.3 nust_varda()

```
void Zmogus::nust_varda (
    std::string vardas) [inline]
```

Nustato žmogaus vardą.

Parametrai

<i>vardas</i>	Žmogaus vardas.
-------------------------------	-----------------

6.4.2.4 pavarde()

```
std::string Zmogus::pavarde () const [inline]
```

Grąžina žmogaus pavardę.

6.4.2.5 vardas()

```
std::string Zmogus::vardas () const [inline]
```

Grąžina žmogaus vardą.

6.4.3 Atributų Dokumentacija

6.4.3.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Žmogaus pavardė.

6.4.3.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Žmogaus vardas.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/zmogus.h](#)

skyrius 7

Failo Dokumentacija

7.1 src/isvesties_pagalb_fjos.cpp Failo Nuoroda

```
#include <iostream>
#include <fstream>
#include <algorithm>
#include <type_traits>
#include "klaidu_valdymas.h"
#include "konstantos_kt_klases.h"
#include "studentas.h"
```

Funkcijos

- std::ofstream [ras_failo_paruosimas](#) (std::string RAS_FAILO_NUORODA)
- bool [pagal_varda_did](#) (Studentas &A, Studentas &B)
- bool [pagal_varda_maz](#) (Studentas &A, Studentas &B)
- bool [pagal_pavarde_did](#) (Studentas &A, Studentas &B)
- bool [pagal_pavarde_maz](#) (Studentas &A, Studentas &B)
- bool [pagal_vidurki_did](#) (Studentas &A, Studentas &B)
- bool [pagal_vidurki_maz](#) (Studentas &A, Studentas &B)
- bool [pagal_mediana_did](#) (Studentas &A, Studentas &B)
- bool [pagal_mediana_maz](#) (Studentas &A, Studentas &B)

7.1.1 Funkcijos Dokumentacija

7.1.1.1 pagal_mediana_did()

```
bool pagal_mediana_did (
    Studentas & A,
    Studentas & B)
```

7.1.1.2 pagal_mediana_maz()

```
bool pagal_mediana_maz (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.3 pagal_pavarde_did()

```
bool pagal_pavarde_did (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.4 pagal_pavarde_maz()

```
bool pagal_pavarde_maz (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.5 pagal_varda_did()

```
bool pagal_varda_did (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.6 pagal_varda_maz()

```
bool pagal_varda_maz (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.7 pagal_vidurki_did()

```
bool pagal_vidurki_did (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.8 pagal_vidurki_maz()

```
bool pagal_vidurki_maz (  
    Studentas & A,  
    Studentas & B)
```

7.1.1.9 ras_failo_paruosimas()

```
std::ofstream ras_failo_paruosimas (  
    std::string RAS_FAILO_NUORODA)
```

7.2 src/isvesties_pagalb_fjos.h Failo Nuoroda

```
#include <algorithm>
#include <type_traits>
#include "studentas.h"
#include "konstantos_kt_klases.h"
```

Funkcijos

- `std::ofstream ras_failo_paruosimas` (`std::string RAS_FAILO_NUORODA`)
- `template<typename T>`
`void stud_rikiavimas (Container(T) &grupe, bool(*rikiavimo_taisykle)(T &, T &))`
- `template<typename T>`
`void rikiavimas (Container(T) &konteineris)`
- `bool pagal_varda_did (Studentas &A, Studentas &B)`
- `bool pagal_varda_maz (Studentas &A, Studentas &B)`
- `bool pagal_pavarde_did (Studentas &A, Studentas &B)`
- `bool pagal_pavarde_maz (Studentas &A, Studentas &B)`
- `bool pagal_vidurki_did (Studentas &A, Studentas &B)`
- `bool pagal_vidurki_maz (Studentas &A, Studentas &B)`
- `bool pagal_mediana_did (Studentas &A, Studentas &B)`
- `bool pagal_mediana_maz (Studentas &A, Studentas &B)`

7.2.1 Funkcijos Dokumentacija

7.2.1.1 pagal_mediana_did()

```
bool pagal_mediana_did (  
    Studentas & A,  
    Studentas & B)
```

7.2.1.2 pagal_mediana_maz()

```
bool pagal_mediana_maz (  
    Studentas & A,  
    Studentas & B)
```

7.2.1.3 pagal_pavarde_did()

```
bool pagal_pavarde_did (  
    Studentas & A,  
    Studentas & B)
```

7.2.1.4 pagal_pavarde_maz()

```
bool pagal_pavarde_maz (  
    Studentas & A,  
    Studentas & B)
```

7.2.1.5 pagal_varda_did()

```
bool pagal_varda_did (
    Studentas & A,
    Studentas & B)
```

7.2.1.6 pagal_varda_maz()

```
bool pagal_varda_maz (
    Studentas & A,
    Studentas & B)
```

7.2.1.7 pagal_vidurki_did()

```
bool pagal_vidurki_did (
    Studentas & A,
    Studentas & B)
```

7.2.1.8 pagal_vidurki_maz()

```
bool pagal_vidurki_maz (
    Studentas & A,
    Studentas & B)
```

7.2.1.9 ras_failo_paruosimas()

```
std::ofstream ras_failo_paruosimas (
    std::string RAS_FAILO_NUORODA)
```

7.2.1.10 rikiavimas()

```
template<typename T>
void rikiavimas (
    Container(T) & konteineris)
```

7.2.1.11 stud_rikiavimas()

```
template<typename T>
void stud_rikiavimas (
    Container(T) & grupe,
    bool(* rikiavimo_taisykle )(T &, T &))
```

7.3 isvesties_pagalb_fjos.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002
00003 #include <algorithm>
00004 #include <type_traits>
00005
00006 #include "studentas.h"
00007 #include "konstantos_kt_klases.h"
00008
00009 // class Studentas;    // forward declaration, vietoje #include "studentas.h", kad išvengt circular
                        // dependency
00010
00011 std::ofstream ras_failo_paruosimas(std::string RAS_FAILO_NUORODA);
00012
00013 // ===== Pagalbinė rikiavimą įvykdanti funkcija (rikiuoja skirtingai, priklausomai nuo to, ar
                        // rikiuojamas vector/deque, ar list) =====
00014
00015 template <typename T> // T = Studentas; be template, kompiliatorius neišvertins constexpr sąlygų ir mes
                        // klaidą, kad pvz. std::deque neturi .sort() metodo
00016 void stud_rikiavimas(Container(T) & grupe, bool (*rikiavimo_taisykle)(T &, T &))
00017 {
00018     // LIST
00019     if constexpr (std::is_same_v<Container(T), std::list<T> >)
00020     {
00021         grupe.sort(rikiavimo_taisykle);
00022     }
00023     // VECTOR / DEQUE
00024     else if constexpr (std::is_same_v<Container(T), Vector<T> > || std::is_same_v<Container(T),
                        std::vector<T> > || std::is_same_v<Container(T), std::deque<T> >)
00025     {
00026         std::sort(grupe.begin(), grupe.end(), rikiavimo_taisykle);
00027     }
00028 }
00029
00030 // APRAŠAS PERKELTAS ČIA, NES TEMPLATE FUNKCIJA (.cpp failan idėjus -- neveikia)
00031 // be nurodytos specif taisyklės, bet kokiam tipui
00032 template <typename T>
00033 void rikiavimas(Container(T) & konteineris)
00034 {
00035     // LIST
00036     if constexpr (std::is_same_v<Container(T), std::list<T> >)
00037     {
00038         konteineris.sort();
00039     }
00040     // VECTOR / DEQUE
00041     else if constexpr (std::is_same_v<Container(T), std::vector<T> > || std::is_same_v<Container(T),
                        std::deque<T> >)
00042     {
00043         std::sort(konteineris.begin(), konteineris.end());
00044     }
00045 }
00046
00047 bool pagal_varda_did(Studentas &A, Studentas &B);
00048 bool pagal_varda_maz(Studentas &A, Studentas &B);
00049 bool pagal_pavarde_did(Studentas &A, Studentas &B);
00050 bool pagal_pavarde_maz(Studentas &A, Studentas &B);
00051 bool pagal_vidurki_did(Studentas &A, Studentas &B);
00052 bool pagal_vidurki_maz(Studentas &A, Studentas &B);
00053 bool pagal_mediana_did(Studentas &A, Studentas &B);
00054 bool pagal_mediana_maz(Studentas &A, Studentas &B);

```

7.4 src/isvestis.cpp Failo Nuoroda

```

#include "studentas.h"
#include "isvestis.h"
#include "isvesties_pagalb_fjos.h"
#include "konstantos_kt_klases.h"
#include "klaidu_valdymas.h"
#include "investies_pagalb_fjos.h"
#include <iostream>
#include <string>
#include <vector>

```

```
#include <deque>
#include <list>
#include <algorithm>
#include <fstream>
#include <iomanip>
```

Funkcijos

- void `isvestis` (std::string RAS_FAILO_NUORODA, `Container(Studentas)` &grupe, `Programos_laikai` &t)
- void `visu_stud_duomenu_generavimo_isvestis` (std::string RAS_FAILO_NUORODA, `Container(Studentas)` &grupe, `Programos_laikai` &t)
- void `skirstoma_isvestis` (std::string RAS_FAILO_NUORODA, `Container(Studentas)` &grupe, `Programos_laikai` &t)
- std::chrono::duration< double > `spausdinimas` (std::string RAS_FAILO_PAV, std::string RAS_FAILO_NUORODA, `Container(Studentas)` &grupe)
- void `studentu_skirstymas` (int strategija, `Container(Studentas)` &grupe, `Container(Studentas)` &blogi, `Container(Studentas)` &geri)

7.4.1 Funkcijos Dokumentacija

7.4.1.1 isvestis()

```
void isvestis (
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.4.1.2 skirstoma_isvestis()

```
void skirstoma_isvestis (
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.4.1.3 spausdinimas()

```
std::chrono::duration< double > spausdinimas (
    std::string RAS_FAILO_PAV,
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe)
```

7.4.1.4 studentu_skirstymas()

```
void studentu_skirstymas (
    int strategija,
    Container(Studentas) & grupe,
    Container(Studentas) & blogi,
    Container(Studentas) & geri)
```

7.4.1.5 visu_stud_duomenu_generavimo_isvestis()

```
void visu_stud_duomenu_generavimo_isvestis (
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.5 src/isvestis.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <chrono>
#include "konstantos_kt_klases.h"
```

Funkcijos

- void `isvestis` (std::string RAS_FAILO_NUORODA, Container(Studentas) &grupe, Programos_laikai &t)
- void `skirstoma_isvestis` (std::string RAS_FAILO_NUORODA, Container(Studentas) &grupe, Programos_laikai &t)
- std::chrono::duration< double > `spausdinimas` (std::string RAS_FAILO_PAV, std::string RAS_FAILO_NUORODA, Container(Studentas) &grupe)
- void `studentu_skirstymas` (int strategija, Container(Studentas) &grupe, Container(Studentas) &blogi, Container(Studentas) &geri)
- void `visu_stud_duomenu_generavimo_isvestis` (std::string RAS_FAILO_NUORODA, Container(Studentas) &grupe, Programos_laikai &t)

7.5.1 Funkcijos Dokumentacija

7.5.1.1 isvestis()

```
void isvestis (
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.5.1.2 skirstoma_isvestis()

```
void skirstoma_isvestis (
    std::string RAS_FAILO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.5.1.3 spausdinimas()

```
std::chrono::duration< double > spausdinimas (
    std::string RAS_FAILLO_PAV,
    std::string RAS_FAILLO_NUORODA,
    Container(Studentas) & grupe)
```

7.5.1.4 studentu_skirstymas()

```
void studentu_skirstymas (
    int strategija,
    Container(Studentas) & grupe,
    Container(Studentas) & blogi,
    Container(Studentas) & geri)
```

7.5.1.5 visu_stud_duomenu_generavimo_isvestis()

```
void visu_stud_duomenu_generavimo_isvestis (
    std::string RAS_FAILLO_NUORODA,
    Container(Studentas) & grupe,
    Programos_laikai & t)
```

7.6 isvestis.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 #include <deque>
00006 #include <list>
00007 #include <chrono>
00008
00009 #include "konstantos_kt_klases.h"
00010 class Studentas;
00011
00012 void isvestis(std::string RAS_FAILLO_NUORODA, Container(Studentas) & grupe, Programos_laikai &t);
00013
00014 void skirstoma_isvestis(std::string RAS_FAILLO_NUORODA, Container(Studentas) & grupe, Programos_laikai
&t);
00015 std::chrono::duration<double> spausdinimas(std::string RAS_FAILLO_PAV, std::string RAS_FAILLO_NUORODA,
Container(Studentas) & grupe);
00016 void studentu_skirstymas(int strategija, Container(Studentas) & grupe, Container(Studentas) & blogi,
Container(Studentas) & geri);
00017
00018 void visu_stud_duomenu_generavimo_isvestis(std::string RAS_FAILLO_NUORODA, Container(Studentas) &
grupe, Programos_laikai &t);
```

7.7 src/ivesties_pagalb_fjos.cpp Failo Nuoroda

```
#include <iostream>
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <sstream>
```

```
#include <fstream>
#include <cstdlib>
#include "ivesties_pagalb_fjos.h"
#include "studentas.h"
#include "klaidu_valdymas.h"
```

Funkcijos

- `std::ifstream skait_failo_paruosimas` (`std::string SK_FAILO_NUORODA`)
- `bool natur_skaiciaus_ivestis` (`int &sk`, `bool(*papild_salygu_netenkinimo_fja)(int)`, `bool ar_ivestis_atsaukiama`)
- `bool vardo_pavardes_ivestis` (`Studentas &A`, `bool ar_ivestis_atsaukiama`)
- `bool ar_sk_ne_tarp_0_ir_10` (`int x`)
- `bool ar_sk_nedidesnis_uz_0` (`int x`)
- `bool ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil` (`int x`)
- `bool ar_sk_nera_1_2_3_4_5_6` (`int x`)

7.7.1 Funkcijos Dokumentacija

7.7.1.1 `ar_sk_ne_tarp_0_ir_10()`

```
bool ar_sk_ne_tarp_0_ir_10 (
    int x)
```

7.7.1.2 `ar_sk_nedidesnis_uz_0()`

```
bool ar_sk_nedidesnis_uz_0 (
    int x)
```

7.7.1.3 `ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil()`

```
bool ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil (
    int x)
```

7.7.1.4 `ar_sk_nera_1_2_3_4_5_6()`

```
bool ar_sk_nera_1_2_3_4_5_6 (
    int x)
```

7.7.1.5 `natur_skaiciaus_ivestis()`

```
bool natur_skaiciaus_ivestis (
    int & sk,
    bool(* papild_salygu_netenkinimo_fja ) (int),
    bool ar_ivestis_atsaukiama)
```

7.7.1.6 skait_failo_paruosimas()

```
std::ifstream skait_failo_paruosimas (
    std::string SK_FAILO_NUORODA)
```

7.7.1.7 vardo_pavardes_ivestis()

```
bool vardo_pavardes_ivestis (
    Studentas & A,
    bool ar_ivestis_atsaukiama)
```

7.8 src/ivesties_pagalb_fjos.h Failo Nuoroda

Funkcijos

- std::ifstream [skait_failo_paruosimas](#) (std::string SK_FAILO_NUORODA)
- bool [vardo_pavardes_ivestis](#) (Studentas &A, bool ar_ivestis_atsaukiama)
- bool [natur_skaiciaus_ivestis](#) (int &sk, bool(*papild_salygu_fja)(int)=nullptr, bool ar_ivestis_atsaukiama=false)
- bool [ar_sk_nedidesnis_uz_0](#) (int x)
- bool [ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil](#) (int x)
- bool [ar_sk_ne_tarp_0_ir_10](#) (int x)
- bool [ar_sk_nera_1_2_3_4_5_6](#) (int x)

7.8.1 Funkcijos Dokumentacija

7.8.1.1 ar_sk_ne_tarp_0_ir_10()

```
bool ar_sk_ne_tarp_0_ir_10 (
    int x)
```

7.8.1.2 ar_sk_nedidesnis_uz_0()

```
bool ar_sk_nedidesnis_uz_0 (
    int x)
```

7.8.1.3 ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil()

```
bool ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil (
    int x)
```

7.8.1.4 ar_sk_nera_1_2_3_4_5_6()

```
bool ar_sk_nera_1_2_3_4_5_6 (
    int x)
```

7.8.1.5 natur_skaiciaus_ivistis()

```
bool natur_skaiciaus_ivistis (
    int & sk,
    bool (* papild_salygu_fja ) (int) = nullptr,
    bool ar_ivistis_atsaukiama = false)
```

7.8.1.6 skait_failo_paruosimas()

```
std::ifstream skait_failo_paruosimas (
    std::string SK_FAILO_NUORODA)
```

7.8.1.7 vardo_pavardes_ivistis()

```
bool vardo_pavardes_ivistis (
    Studentas & A,
    bool ar_ivistis_atsaukiama)
```

7.9 ivesties_pagalb_fjos.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002
00003 class Studentas; // forward declaration, vietoje #include "studentas.h", kad išvengt circular
00004 // dependency
00005 // ivesties (skaitymo) failo pavadinimo gavimo funkcija
00006 std::ifstream skait_failo_paruosimas(std::string SK_FAILO_NUORODA);
00007
00008 // pagalbinės vartotojo ivesties programoj funkcijos
00009 bool vardo_pavardes_ivistis(Studentas &A, bool ar_ivistis_atsaukiama);
00010 bool natur_skaiciaus_ivistis(int &sk, bool (*papild_salygu_fja) (int) = nullptr, bool
00011 // ar_ivistis_atsaukiama = false);
00012 // pagalbinės ivesties sąlygų tikrinimo funkcijos
00013 bool ar_sk_nedidesnis_uz_0(int x);
00014 bool ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil(int x);
00015 bool ar_sk_ne_tarp_0_ir_10(int x);
00016 bool ar_sk_nera_1_2_3_4_5_6(int x);
```

7.10 src/ivistis.cpp Failo Nuoroda

```
#include <iostream>
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <algorithm>
#include <iomanip>
#include <limits>
#include <cstdlib>
#include <sstream>
#include <chrono>
#include <fstream>
#include <random>
```

```
#include "investis.h"
#include "investies_pagalb_fjos.h"
#include "konstantos_kt_klases.h"
#include "klaidu_valdymas.h"
#include "studentas.h"
#include "zmogus.h"
```

Funkcijos

- void [failo_investis](#) (std::ifstream &sk_failas, [Container\(Studentas\)](#) &grupe, [Programos_laikai](#) &t)
- void [rank_investis](#) ([Container\(Studentas\)](#) &grupe)
- void [misri_investis](#) ([Container\(Studentas\)](#) &grupe)
- void [generuota_investis](#) ([Container\(Studentas\)](#) &grupe)
- void [visu_stud_duomenu_generavimo_investis](#) ([Container\(Studentas\)](#) &grupe)

7.10.1 Funkcijos Dokumentacija

7.10.1.1 failo_investis()

```
void failo_investis (
    std::ifstream & sk_failas,
    Container\(Studentas\) & grupe,
    Programos\_laikai & t)
```

7.10.1.2 generuota_investis()

```
void generuota_investis (
    Container\(Studentas\) & grupe)
```

7.10.1.3 misri_investis()

```
void misri_investis (
    Container\(Studentas\) & grupe)
```

7.10.1.4 rank_investis()

```
void rank_investis (
    Container\(Studentas\) & grupe)
```

7.10.1.5 visu_stud_duomenu_generavimo_investis()

```
void visu_stud_duomenu_generavimo_investis (
    Container\(Studentas\) & grupe)
```

7.11 src/ivestis.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <deque>
#include <list>
#include "konstantos_kt_klases.h"
```

Funkcijos

- void [failo_ivestis](#) (std::ifstream &sk_failas, [Container\(Studentas\)](#) &grupe, [Programos_laikai](#) &t)
- void [rank_ivestis](#) ([Container\(Studentas\)](#) &grupe)
- void [misri_ivestis](#) ([Container\(Studentas\)](#) &grupe)
- void [generuota_ivestis](#) ([Container\(Studentas\)](#) &grupe)
- void [visu_stud_duomenu_generavimo_ivestis](#) ([Container\(Studentas\)](#) &grupe)

7.11.1 Funkcijos Dokumentacija

7.11.1.1 failo_ivestis()

```
void failo_ivestis (
    std::ifstream & sk_failas,
    Container\(Studentas\) & grupe,
    Programos\_laikai & t)
```

7.11.1.2 generuota_ivestis()

```
void generuota_ivestis (
    Container\(Studentas\) & grupe)
```

7.11.1.3 misri_ivestis()

```
void misri_ivestis (
    Container\(Studentas\) & grupe)
```

7.11.1.4 rank_ivestis()

```
void rank_ivestis (
    Container\(Studentas\) & grupe)
```

7.11.1.5 visu_stud_duomenu_generavimo_ivestis()

```
void visu_stud_duomenu_generavimo_ivestis (
    Container\(Studentas\) & grupe)
```

7.12 igestis.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 #include <deque>
00006 #include <list>
00007
00008 #include "konstantos_kt_klases.h"
00009 class Studentas;
00010
00011 void failo_igestis(std::ifstream &sk_failas, /*const std::string SK_FAILO_NUORODA,*/
00012 Container(Studentas) &grupe, Programos_laikai &t);
00012 void rank_igestis(Container(Studentas) &grupe);
00013 void misri_igestis(Container(Studentas) &grupe);
00014 void generuota_igestis(Container(Studentas) &grupe);
00015 void visu_stud_duomenu_generavimo_igestis(Container(Studentas) &grupe);
00016 // void generuota_skirstoma_pazymiu_igestis(std::vector<Studentas> &geri, std::vector<Studentas>
00017 &blogi);
```

7.13 src/klaidu_valdymas.cpp Failo Nuoroda

```
#include <iostream>
#include "klaidu_valdymas.h"
```

Funkcijos

- void [igesties_klaidos_valdymas](#) ()

7.13.1 Funkcijos Dokumentacija

7.13.1.1 igesties_klaidos_valdymas()

```
void igesties_klaidos_valdymas ()
```

7.14 src/klaidu_valdymas.h Failo Nuoroda

Funkcijos

- void [igesties_klaidos_valdymas](#) ()

7.14.1 Funkcijos Dokumentacija

7.14.1.1 igesties_klaidos_valdymas()

```
void igesties_klaidos_valdymas ()
```

7.15 klaidu_valdymas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002
00003 void ivesties_klaidos_valdymas();
```

7.16 src/konstantos_kt_klases.h Failo Nuoroda

```
#include "vector.h"
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <chrono>
#include <limits>
```

Klasės

- class [Programos_laikai](#)

Apibrėžimai

- #define [Container](#)(T)

Kintamieji

- const auto [MAX_INT](#) = std::numeric_limits<int>::max()

7.16.1 Apibrėžimų Dokumentacija

7.16.1.1 Container

```
#define Container(  
    T)
```

Reikšmė:

[Vector<T>](#)

7.16.2 Kintamojo Dokumentacija

7.16.2.1 MAX_INT

```
const auto MAX_INT = std::numeric_limits<int>::max()
```

7.17 konstantos_kt_klases.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002
00003 #include "vector.h"
00004
00005 #include <string>
00006 #include <vector>
00007 #include <deque>
00008 #include <list>
00009 #include <chrono>
00010 #include <limits> // maksimaliai int reikšmei gauti
00011
00012 // KONTEINERIO TIPAS, KURĮ NAUDOJA VISA PROGRAMA (VECTOR / DEQUE / LIST)
00013 #define Container(T) Vector<T>
00014
00015 // kad būtų trumpiau
00016 const auto MAX_INT = std::numeric_limits<int>::max();
00017
00018 class Programos_laikai
00019 {
00020 public:
00021     // failo apdorojimo laikai
00022     std::chrono::duration<double> duomenu_nuskaitymas;
00023     std::chrono::duration<double> duomenu_rikiavimas;
00024     std::chrono::duration<double> studentu_skirstymas;
00025     // std::chrono::duration<double> geru_isvedimas;
00026     // std::chrono::duration<double> blogu_isvedimas;
00027     // std::chrono::duration<double> visa_trukme = std::chrono::milliseconds::zero();
00028     std::chrono::duration<double> visa_trukme_su_ivestim = std::chrono::milliseconds::zero();
00029
00030     // failo sukūrimo (su visais stud. duomenimis) laikai
00031     // std::chrono::duration<double> failo_generavimo_trukme = std::chrono::milliseconds::zero();
00032
00033     // metodai
00034     void spausd_laikus();
00035 };
```

7.18 src/kt_klasu_metodai.cpp Failo Nuoroda

```
#include <iostream>
#include <vector>
#include <deque>
#include <list>
#include <algorithm>
#include "konstantos_kt_klases.h"
```

7.19 src/pagr.cpp Failo Nuoroda

```
#include <iostream>
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <chrono>
#include <fstream>
#include <typeinfo>
#include "konstantos_kt_klases.h"
#include "studentas.h"
#include "ivestis.h"
#include "isvestis.h"
#include "ivesties_pagalb_fjos.h"
```

Funkcijos

- int `main` ()

7.19.1 Funkcijos Dokumentacija

7.19.1.1 main()

```
int main ()
```

7.20 src/studentas.cpp Failo Nuoroda

```
#include "studentas.h"
```

Funkcijos

- std::ostream & `operator<<` (std::ostream &os, const `Studentas` &stud)
- std::istream & `operator>>` (std::istream &is, `Studentas` &stud)

7.20.1 Funkcijos Dokumentacija

7.20.1.1 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & stud)
```

Parametrai

<i>os</i>	Nuoroda į išvesties srautą.
<i>stud</i>	Nuoroda į išvesimą studentą.

Gražina

std::ostream& nuoroda į išvesties srautą (tinkamam operatoriaus veikimui).

7.20.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & stud)
```

Parametrai

<i>is</i>	Nuoroda į įvesties srautą.
<i>stud</i>	Nuoroda į studento objektą, į kurį bus nuskaitomi duomenys.

Gražina

std::istream& Gražinama nuoroda į įvesties srautą (tinkamam operatoriaus veikimui).

7.21 src/studentas.h Failo Nuoroda

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <fstream>
#include "zmogus.h"
#include "investies_pagalb_fjos.h"
#include "isvesties_pagalb_fjos.h"
#include "konstantos_kt_klases.h"
#include "klaidu_valdymas.h"
```

Klasės

- class [Studentas](#)

7.22 studentas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005 #include <sstream>
00006 #include <iomanip>
00007 #include <fstream>
00008
00009 #include "zmogus.h"
00010 #include "investies_pagalb_fjos.h"
00011 #include "isvesties_pagalb_fjos.h"
00012 #include "konstantos_kt_klases.h"
00013 #include "klaidu_valdymas.h"
00014
00015 class Studentas : public Zmogus
00016 {
00017 private:
00018     Container(int) pazymiai_;
00019     int egzo_rezas_;
00020     float rezas_vid_;
00021     float rezas_med_;
```

```

00027     inline static std::string galut_; // "v" / "m" // be inline -- klaida: "undefined reference to
Studentas::galut_"
00028
00029 public:
00030     // nuskaitant
00031     Studentas() : Zmogus(), egzo_rezas_(0), rezas_vid_(0), rezas_med_(0) {}
00032     // surasant rankiniu budu
00033     Studentas(std::string vardas, std::string pavarde) : Zmogus(vardas, pavarde), egzo_rezas_(0),
rezas_vid_(0), rezas_med_(0) {}
00034
00035     // kopijavimo konstr [Studentas x(y) / Studentas x = y]
00036     Studentas(const Studentas &kitas)
00037     : Zmogus(kitas),
00038       pazymiai_(kitas.pazymiai_),
00039       egzo_rezas_(kitas.egzo_rezas_),
00040       rezas_vid_(kitas.rezas_vid_),
00041       rezas_med_(kitas.rezas_med_) {}
00042     // perkélimo konstr [Studentas x(std::move(y)) / Studentas x = std::move(y)]
00043     Studentas(Studentas &&kitas)
00044     : Zmogus(std::move(kitas)),
00045       pazymiai_(std::move(kitas.pazymiai_)),
00046       egzo_rezas_(kitas.egzo_rezas_), // be std::move, nes paprastiem tipam kaip int/float tai
netikslinga (skirtingai nei std::string, std::vector -- objektam)
00047     rezas_vid_(kitas.rezas_vid_),
00048     rezas_med_(kitas.rezas_med_)
00049     {
00050         kitas.egzo_rezas_ = 0;
00051         kitas.rezas_vid_ = 0;
00052         kitas.rezas_med_ = 0;
00053     }
00054     // destruktorius
00055     ~Studentas()
00056     {
00057         pazymiai_.clear();
00058     }
00059     // perkrauti operatoriai
00060
00061     // kopijavimo priskyrimo operatorius [x = y]
00062     Studentas &operator=(const Studentas &kitas);
00063     // perkélimo priskyrimo operatorius [x = std::move(y)]
00064     Studentas &operator=(Studentas &&kitas);
00065
00066     friend std::ostream &operator<<(std::ostream &os, const Studentas &stud);
00067     friend std::istream &operator>>(std::istream &is, Studentas &stud);
00068
00069     // gavikai / getteriai
00070
00071     inline Container(int) pazymiai() const { return pazymiai_; }
00072     inline int egzo_rezas() const { return egzo_rezas_; }
00073     inline float rezas_vid() const { return rezas_vid_; }
00074     inline float rezas_med() const { return rezas_med_; }
00075     inline size_t pazymiu_sk() { return pazymiai_.size(); }
00076
00077     static std::string galut() { return galut_; }
00078
00079     // nustatytojai / setteriai
00080
00081     bool invest_varda_pavarde(bool ar_investis_atsaukiama = false) override;
00082
00083     void pridet_pazymi(int paz);
00084     void nust_egzo_reza(int rez);
00085     void invest_egzo_reza();
00086     void apsk_vid();
00087     void apsk_med();
00088
00089     static void nust_galutinio_tipa(std::string v_m);
00090
00091     // kt metodai
00092
00093     void uzpildyt_pazymius_iki_min(int min_pazymiu_sk);
00094 };

```

7.23 src/vector.h Failo Nuoroda

```

#include <cstdint>
#include <algorithm>
#include <vector>
#include <stdexcept>

```

```
#include <string>
#include <new>
#include <memory>
```

Klasės

- class `Vector< T >`

Funkcijos

- `template<typename T>`
`bool operator== (const Vector< T > &lhs, const Vector< T > &rhs)`
Patikrina, ar du vektoriai yra lygūs.
- `template<typename T>`
`bool operator!= (const Vector< T > &lhs, const Vector< T > &rhs)`
Patikrina, ar du vektoriai yra nelygūs.
- `template<typename T>`
`void swap (Vector< T > &lhs, Vector< T > &rhs) noexcept`
Funkcija dviejų vektorių turiniui sukeisti.
- `template<typename T, typename U>`
`constexpr size_t erase (Vector< T > &vector, const U &value)`
Ištrina iš vektoriaus visus elementus, lygius nurodytai reikšmei. Funkcija perkelia visus paliekamus elementus į vektoriaus priekį ir tuomet ištrina likusius elementus gale.
- `template<typename T, typename Pred>`
`constexpr size_t erase_if (Vector< T > &vector, Pred pred)`
Ištrina iš vektoriaus visus elementus, atitinkančius nurodytą sąlygą (predikatą).

7.23.1 Funkcijos Dokumentacija

7.23.1.1 erase()

```
template<typename T, typename U>
size_t erase (
    Vector< T > & vector,
    const U & value) [constexpr]
```

Ištrina iš vektoriaus visus elementus, lygius nurodytai reikšmei. Funkcija perkelia visus paliekamus elementus į vektoriaus priekį ir tuomet ištrina likusius elementus gale.

Template Parameters

<i>T</i>	Vektoriaus elementų tipas.
<i>U</i>	Ieškomos reikšmės tipas.

Parametrai

<i>vector</i>	Vektorius, iš kurio šalinami elementai.
---------------	---

<i>value</i>	Reikšmė, kurios visus atitikmenis reikia ištrinti.
--------------	--

Gražina

Bendras ištrintų elementų skaičius.

7.23.1.2 erase_if()

```
template<typename T, typename Pred>
size_t erase_if (
    Vector< T > & vector,
    Pred pred) [constexpr]
```

Ištrina iš vektoriaus visus elementus, atitinkančius nurodytą sąlygą (predikatą).

Veikimo principas toks pat kaip `erase` funkcijos, tačiau šalinami tie elementai, kuriems predikato funkcija grąžina `true`.

Template Parameters

<i>T</i>	Vektoriaus elementų tipas.
<i>Pred</i>	Predikato tipas (pvz., lambda ar paprasta funkcija).

Parametrai

<i>vector</i>	Vektorius, iš kurio šalinami elementai.
<i>pred</i>	Sąlygos funkcija. Vieną argumentą priimanti funkcija, grąžinanti <code>bool</code> .

Gražina

Bendras ištrintų elementų skaičius.

7.23.1.3 operator!=()

```
template<typename T>
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Patikrina, ar du vektoriai yra nelygūs.

Operatorius atvirkščias operatoriui `operator==`.

Template Parameters

<i>T</i>	Vektoriuose saugomų elementų tipas.
----------	-------------------------------------

Parametrai

<i>lhs</i>	Kairysis vektorius.
<i>rhs</i>	Dešinysis vektorius.

Gražina

`true`, jei vektoriai skiriasi dydžiu arba bent vienu elementu, kitu atveju `false`.

7.23.1.4 operator==()

```
template<typename T>
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

Patikrina, ar du vektoriai yra lygūs.

Vektoriai laikomi lygiais, jei jų dydžiai (`size()`) sutampa ir visi atitinkamose pozicijose esantys elementai yra lygūs.

Template Parameters

<i>T</i>	Vektoriuose saugomų elementų tipas.
----------	-------------------------------------

Parametrai

<i>lhs</i>	Kairysis vektorius.
<i>rhs</i>	Dešinysis vektorius.

Gražina

`true`, jei vektoriai visiškai sutampa, kitu atveju `false`.

7.23.1.5 swap()

```
template<typename T>
void swap (
    Vector< T > & lhs,
    Vector< T > & rhs) [noexcept]
```

Funkcija dviejų vektorių turiniui sukeisti.

Funkcija viduje taiko `Vector` klasės `swap (Vector &)` metodą.

Template Parameters

<i>T</i>	Vektorių elementų tipas.
----------	--------------------------

Parametrai

<i>lhs</i>	Pirmasis vektorius.
<i>rhs</i>	Antrasis vektorius.

7.24 vector.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #pragma once
00002
00003 #include <cstddef>
00004 #include <algorithm>
00005 #include <vector> // pavyzdžiui
00006 #include <stdexcept>
00007 #include <string>
00008 #include <new> // dėl placement new
00009 #include <memory> // dėl std::uninitialized_..., std::destroy
00010
00011 // klasė šabloninė, tai lieka .h
00012
00013 template <typename T>
00014 class Vector
00015 {
00016 public:
00017     // TIPAI (MEMBER TYPES)
00018
00019     using value_type = T;
00020     // allocator -- atskiro nedarysma
00021     using size_type = std::size_t;
00022     using difference_type = std::ptrdiff_t;
00023
00024     using reference = value_type &;
00025     using const_reference = const value_type &;
00026
00027     using pointer = value_type *; // būtų allocator čia, bet jei jo nedarom, tai taip
00028     using const_pointer = const value_type *;
00029
00030     using iterator = pointer;
00031     using const_iterator = const_pointer;
00032     using reverse_iterator = std::reverse_iterator<iterator>;
00033     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00034
00035 private:
00036     T *data_;
00037     size_type size_;
00038     size_type capacity_;
00039
00040 public:
00041     // METODAI (MEMBER FUNCTIONS)
00042
00043     // ===== konstruktoriai
00044
00045     // numatytais konstruktorius
00046     Vector() : data_(nullptr), size_(0), capacity_(0) {}
00047     // konstruktorius n elementų vektoriaus
00048     Vector(size_type n) : size_(n), capacity_(n)
00049     {
00050         data_ = static_cast<T *>(operator new(n * sizeof(T))); // paskiriam gryną neužimtą dinaminę
00051         atmintį (BE objektų konstravimo)
00052         std::uninitialized_value_construct_n(data_, n); // sukonstruojam objektus default
00053         reikšmėmis paskirtoje neužimtoje atmintyje
00054     }
00055     // konstruktorius su visų prad. elementų užpildymu elementais x
00056     Vector(size_type n, const T &x) : size_(n), capacity_(n)
00057     {
00058         data_ = static_cast<T *>(operator new(n * sizeof(T))); // paskiriam gryną neužimtą dinaminę
00059         atmintį (BE objektų konstravimo)
00060         std::uninitialized_fill_n(data_, n, x); // užpildo neužimtą atmintį reikšmėmis
00061         (objektais) x iki n-tojo (size_-tojo) elemento
00062     }
00063     // konstruktorius su inicializavimo sąrašu

```

```

00074     Vector(std::initializer_list<T> list) : size_(list.size()), capacity_(list.size())
00075     {
00076         data_ = static_cast<T *>(operator new(list.size() * sizeof(T))); // paskiriam gryną neužimtą
dinaminę atmintį (BE objektų konstravimo)
00077         std::uninitialized_copy(list.begin(), list.end(), data_);
00078     }
00079     // kopijavimo konstruktorius
00082     Vector(const Vector &other) : size_(other.size_), capacity_(other.size_)
00083     {
00084         data_ = static_cast<T *>(operator new(other.size_ * sizeof(T))); // paskiriam gryną neužimtą
dinaminę atmintį (BE objektų konstravimo)
00085         std::uninitialized_copy(other.data_, other.data_ + size_, data_);
00086     }
00087     // perkėlimo konstruktorius
00092     Vector(Vector &&other) noexcept : data_(other.data_), size_(other.size_), capacity_(other.size_)
// be noexcept neveiks perkėlimas, vyks kopijavimas tsg
00093     {
00094         // AR TSG DESTRUKTORIUM?
00095         other.data_ = nullptr;
00096         other.size_ = 0;
00097         other.capacity_ = 0;
00098     }
00099     ~Vector()
00100     {
00101         std::destroy(data_, data_ + size_); // iškviečia visų masyvo data_ objektų destruktorius
00102         size_ = 0;
00103         capacity_ = 0;
00104         operator delete(data_); // atlaisvina data_ atmintį
00105         data_ = nullptr;
00106     }
00107     // ===== priskyrimo operatoriai
00108     // kopijavimo priskyrimo operatorius
00109     Vector &operator=(const Vector &other)
00110     {
00111         if (this == &other)
00112             return *this;
00113
00114         T *temp;
00115
00116         if (other.data_) // tikrinam, kad nereiktų priskirti new T[0] ar kopijuot nullptr turinį
00117         {
00118             temp = static_cast<T *>(operator new(other.capacity_ * sizeof(T)));
00119             std::uninitialized_copy(other.data_, other.data_ + other.size_, temp);
00120         }
00121         else
00122         {
00123             temp = nullptr;
00124         }
00125
00126         std::destroy(data_, data_ + size_); // operator delete pats nekviečia destruktorių, dėl to
turim juos išsikviest patys prieš laisvinant atmintį
00127         operator delete(data_); // ištrinam PO atminties (galimo) priskyrimo, kad jeigu
netyčia nepavyktų jos paskirti, duomenys nepradingtų
00128         data_ = temp;
00129         size_ = other.size_;
00130         capacity_ = other.capacity_;
00131
00132         return *this;
00133     }
00134     // perkėlimo priskyrimo operatorius
00135     Vector &operator=(Vector &&other) noexcept // be noexcept neveiks perkėlimas, vyks kopijavimas tsg
00136     {
00137         if (this == &other)
00138             return *this;
00139
00140         std::destroy(data_, data_ + size_);
00141         operator delete(data_);
00142
00143         data_ = other.data_;
00144         size_ = other.size_;
00145         capacity_ = other.capacity_;
00146
00147         other.data_ = nullptr;
00148         other.size_ = 0;
00149         other.capacity_ = 0;
00150
00151         return *this;
00152     }
00153     // ===== elementų pasiekimo metodai
00154     reference at(size_type i) // size_type -- neigiamas nebus
00155     {

```

```

00180         if (i >= size_)
00181         {
00182             throw std::out_of_range("Vector::at() : Index " + std::to_string(i) + " is out of range
(size is " + std::to_string(size_) + ")");
00183         }
00184         return data_[i];
00185     }
00186
00193     const_reference at(size_type i) const // size_type -- neigiamas nebus
00194     {
00195         if (i >= size_)
00196         {
00197             throw std::out_of_range("Vector::at() const : Index " + std::to_string(i) + " is out of
range (size is " + std::to_string(size_) + ")");
00198         }
00199         return data_[i];
00200     }
00201
00208     reference operator[](size_type i)
00209     {
00210         return data_[i];
00211     }
00212
00219     const_reference operator[](size_type i) const
00220     {
00221         return data_[i];
00222     }
00223
00227     reference front()
00228     {
00229         return data_[0];
00230     }
00231
00235     const_reference front() const
00236     {
00237         return data_[0];
00238     }
00239
00243     reference back()
00244     {
00245         return data_[size_ - 1];
00246     }
00247
00251     const_reference back() const
00252     {
00253         return data_[size_ - 1];
00254     }
00255
00258     T *data()
00259     {
00260         return data_;
00261     }
00262
00263     // ===== iteratoriai
00264
00267     iterator begin()
00268     {
00269         return data_;
00270     }
00271
00274     const_iterator begin() const // kaipo perdengimas
00275     {
00276         return data_;
00277     }
00278
00284     const_iterator cbegin() const
00285     {
00286         return data_;
00287     }
00288
00291     iterator end()
00292     {
00293         return data_ + size_;
00294     }
00295
00298     const_iterator end() const // kaipo perdengimas
00299     {
00300         return data_ + size_;
00301     }
00302
00305     const_iterator cend() const
00306     {
00307         return data_ + size_;
00308     }
00309
00312     reverse_iterator rbegin()
00313     {

```

```

00314         return std::reverse_iterator(data_ + size_);
00315     }
00316
00319     const_reverse_iterator rbegin() const
00320     {
00321         return std::reverse_iterator(data_ + size_);
00322     }
00323
00326     const_reverse_iterator crbegin() const
00327     {
00328         return std::reverse_iterator(data_ + size_);
00329     }
00330
00333     reverse_iterator rend()
00334     {
00335         return std::reverse_iterator(data_);
00336     }
00337
00340     const_reverse_iterator rend() const
00341     {
00342         return std::reverse_iterator(data_);
00343     }
00344
00347     const_reverse_iterator crend() const
00348     {
00349         return std::reverse_iterator(data_);
00350     }
00351
00352     // ===== talpa
00353
00358     bool empty() const
00359     {
00360         return begin() == end();
00361     }
00362
00365     size_type size() const
00366     {
00367         return size_;
00368     }
00369
00375     void reserve(size_type new_cap)
00376     {
00377         if (new_cap <= capacity_)
00378             return; // taip elgiasi std::vector::reserve -- jei prašoma rezervuoti mažiau nei dab.
00379         capacity_, tai funkcija nieko nedaro
00380
00380         T *new_alloc = static_cast<T *>(operator new(new_cap * sizeof(T)));
00381         std::uninitialized_move(begin(), end(), new_alloc);
00382
00383         std::destroy(begin(), end()); // net ir po move, objektai nėra sunaikinti
00384         operator delete(data_);
00385
00386         data_ = new_alloc;
00387         capacity_ = new_cap;
00388     }
00389
00395     size_type capacity() const
00396     {
00397         return capacity_;
00398     }
00399
00403     void shrink_to_fit()
00404     {
00405         if (capacity_ == size_)
00406             return;
00407
00408         T *new_alloc = static_cast<T *>(operator new(size_ * sizeof(T)));
00409         std::uninitialized_move(begin(), end(), new_alloc);
00410
00411         std::destroy(begin(), end());
00412         operator delete(data_);
00413
00414         data_ = new_alloc;
00415         capacity_ = size_;
00416     }
00417
00418     // ===== keitimo/modifikavimo metodai
00419
00425     void clear()
00426     {
00427         std::destroy(begin(), end());
00428         size_ = 0;
00429     }
00430
00437     void push_back(const T &value)
00438     {
00439         if (size_ == capacity_)

```

```

00440     {
00441         if (capacity_ == 0)
00442             reserve(1);
00443         else
00444             reserve(capacity_ * 2);
00445     }
00446
00447     std::construct_at(data_ + size_, value);
00448     size_++;
00449 }
00450
00451 void push_back(T &&value)
00452 {
00453     if (size_ == capacity_)
00454     {
00455         if (capacity_ == 0)
00456             reserve(1);
00457         else
00458             reserve(capacity_ * 2);
00459     }
00460
00461     std::construct_at(data_ + size_, std::move(value));
00462     size_++;
00463 }
00464
00465 void pop_back()
00466 {
00467     if (size_ > 0)
00468     {
00469         std::destroy_at(data_ + size_ - 1);
00470         size_--;
00471     }
00472 }
00473
00474 iterator insert(const_iterator pos, const T &value) // value -- const, tai negalėsime
std::move(value)
00475 {
00476     size_type idx = pos - begin();
00477
00478     if (size_ == capacity_)
00479     {
00480         if (capacity_ == 0)
00481             reserve(1);
00482         else
00483             reserve(capacity_ * 2); // po reserve, iteratorius pos nebegalioja
00484     }
00485
00486     if (idx == size_)
00487     {
00488         std::construct_at(data_ + size_, value);
00489     }
00490     else
00491     {
00492         std::construct_at(data_ + size_, std::move(data_[size_ - 1])); // paskutinis elementas
vienintelis turi būti perkeltas nenaudojamon atmintin, taigi čia padarom atskirai
00493         std::move_backward(data_ + idx, data_ + size_ - 1, data_ + size_);
00494         data_[idx] = value;
00495     }
00496     size_++;
00497
00498     return begin() + idx;
00499 }
00500
00501 iterator insert(const_iterator pos, T &&value)
00502 {
00503     size_type idx = pos - begin();
00504
00505     if (size_ == capacity_)
00506     {
00507         if (capacity_ == 0)
00508             reserve(1);
00509         else
00510             reserve(capacity_ * 2); // po reserve, iteratorius pos nebegalioja
00511     }
00512
00513     if (idx == size_)
00514     {
00515         std::construct_at(data_ + size_, std::move(value));
00516     }
00517     else
00518     {
00519         std::construct_at(data_ + size_, std::move(data_[size_ - 1])); // paskutinis elementas
vienintelis turi būti perkeltas nenaudojamon atmintin, taigi čia padarom atskirai
00520         std::move_backward(data_ + idx, data_ + size_ - 1, data_ + size_);
00521         data_[idx] = std::move(value);
00522     }
00523     size_++;
00524 }
00525

```

```

00551         return begin() + idx;
00552     }
00561     iterator insert(const_iterator pos, size_type count, const T &value)
00562     {
00563         size_type idx = pos - begin();
00564
00565         if (count == 0)
00566             return begin() + idx; // pos -- const, tai gal neleis grąžint jo tiesiogiai (grąžina ne
const)
00567
00568         size_type new_size = size_ + count;
00569         if (new_size > capacity_)
00570         {
00571             if (capacity_ == 0)
00572             {
00573                 reserve(count);
00574             }
00575             else
00576             {
00577                 size_type new_cap = capacity_;
00578                 while (new_cap < new_size)
00579                     new_cap *= 2;
00580                 reserve(new_cap);
00581             }
00582         }
00583
00584         if (idx == size_)
00585         {
00586             std::uninitialized_fill_n(data_ + size_, count, value); // naudojam tai, o ne push_back(),
kad išvengt nebereikalingų capacity_ patikrų (push_back kas kart tai tikrina, bet mes gi jau
patikrinom)
00587         }
00588         else
00589         {
00590             if (size_ - idx > count) // jei įterpsimų elementų sk. yra mažesnis už pastumsimų elementų
skaičių (pvz. prieš 3-ąjį nuo galo elementą įterpiant 1-2 elementus)
00591             {
00592                 // čia taip perkeliame tuos elementus, kurie atsidurs DAR NELIESTOJ atminty
00593                 std::uninitialized_move(data_ + size_ - count, data_ + size_, data_ + size_);
00594                 // čia taip perkeliame tuos elementus, kurie atsidurs JAU LIESTOJ atminty
00595                 std::move_backward(data_ + idx, data_ + size_ - count, data_ + size_);
00596                 // čia pagaliau įterpiame count skaičių elementų value prieš buvusį idx-tąjį elementą
(buvusį adresu pos)
00597                 std::fill(data_ + idx, data_ + idx + count, value);
00598             }
00599             else // jei įterpsimų elementų sk. yra lygus/didesnis už pastumsimų elementų skaičių
(pvz., prieš 3-ąjį nuo galo elementą įterpiant 3+ elementus)
00600             {
00601                 // čia taip įterpiame tuos elementus value, kurie atsidurs DAR NELIESTOJ atminty
00602                 std::uninitialized_fill_n(data_ + size_, idx + count - size_, value); // jeigu size_ -
idx == count, tai čia niekas neįvyks
00603                 // čia taip perkeliame perkeltinus elementus (dar nelieston atmintin)
00604                 std::uninitialized_move(data_ + idx, data_ + size_, data_ + idx + count);
00605                 // čia taip įterpiame tuos elementus value, kurie atsidurs JAU LIESTOJ atminty
00606                 std::fill(data_ + idx, data_ + size_, value);
00607             }
00608         }
00609
00610         size_ = new_size;
00611
00612         return begin() + idx;
00613     }
00614
00621     iterator erase(iterator pos)
00622     {
00623         size_type idx = pos - begin();
00624         for (size_type i = idx; i < size_ - 1; i++)
00625         {
00626             data_[i] = std::move(data_[i + 1]);
00627         }
00628         std::destroy_at(data_ + size_ - 1); // ištrinam paskutinį elementą (dabar jis dubliuojamas su
priešpaskutiniu)
00629         size_--;
00630         return begin() + idx; // == pos
00631     }
00632
00639     iterator erase(const_iterator pos)
00640     {
00641         size_type idx = pos - begin();
00642         for (size_type i = idx; i < size_ - 1; i++)
00643         {
00644             data_[i] = std::move(data_[i + 1]);
00645         }
00646         std::destroy_at(data_ + size_ - 1); // ištrinam paskutinį elementą (dabar jis dubliuojamas su
priešpaskutiniu)
00647         size_--;
00648         return begin() + idx; // == pos

```

```

00649     }
00650
00651     iterator erase(iterator first, iterator last)
00652     {
00653         size_type first_idx = first - begin();
00654         size_type count = last - first;
00655
00656         if (count == 0)
00657             return begin() + first_idx; // == first
00658
00659         for (size_type i = first_idx; i < size_ - count; i++)
00660         {
00661             data_[i] = std::move(data_[i + count]);
00662         }
00663         std::destroy(data_ + (size_ - count), data_ + size_); // ištrinam paskutinius count elementų
00664         (dbr jie dubliuoja, nebereikalingi)
00665
00666         size_ -= count;
00667
00668         return begin() + first_idx;
00669     }
00670
00671     iterator erase(const_iterator first, const_iterator last)
00672     {
00673         size_type first_idx = first - begin();
00674         size_type count = last - first;
00675
00676         if (count == 0)
00677             return begin() + first_idx; // == first
00678
00679         for (size_type i = first_idx; i < size_ - count; i++)
00680         {
00681             data_[i] = std::move(data_[i + count]);
00682         }
00683         std::destroy(data_ + (size_ - count), data_ + size_); // ištrinam paskutinius count elementų
00684         (dbr jie dubliuoja, nebereikalingi)
00685
00686         size_ -= count;
00687
00688         return begin() + first_idx;
00689     }
00690
00691     void resize(size_type count)
00692     {
00693         if (count == size_)
00694         {
00695             return;
00696         }
00697         if (count < size_)
00698         {
00699             std::destroy(data_ + count, data_ + size_);
00700         }
00701         else
00702         {
00703             if (capacity_ < count)
00704             {
00705                 size_type new_cap = capacity_;
00706                 while (new_cap < count)
00707                     new_cap *= 2;
00708                 reserve(new_cap);
00709             }
00710             std::uninitialized_value_construct_n(data_ + size_, count - size_);
00711         }
00712         size_ = count;
00713     }
00714
00715     void resize(size_type count, const T &value)
00716     {
00717         if (count == size_)
00718         {
00719             return;
00720         }
00721         if (count < size_)
00722         {
00723             std::destroy(data_ + count, data_ + size_);
00724         }
00725         else
00726         {
00727             if (capacity_ < count)
00728             {
00729                 size_type new_cap = capacity_;
00730                 while (new_cap < count)
00731                     new_cap *= 2;
00732                 reserve(new_cap);
00733             }
00734             std::uninitialized_fill_n(data_ + size_, count - size_, value);
00735         }
00736     }

```

```

00759         size_ = count;
00760     }
00761
00766     void swap(Vector &other) noexcept
00767     {
00768         std::swap(data_, other.data_);
00769         std::swap(size_, other.size_);
00770         std::swap(capacity_, other.capacity_);
00771     }
00772
00773     // ===== lyginimo operatoriai
00774 };
00775
00783 template <typename T>
00784 bool operator==(const Vector<T> &lhs, const Vector<T> &rhs)
00785 {
00786     if (lhs.size() != rhs.size())
00787         return false;
00788     for (size_t i = 0; i < lhs.size(); i++)
00789     {
00790         if (lhs[i] != rhs[i])
00791             return false;
00792     }
00793     return true;
00794 }
00795
00803 template <typename T>
00804 bool operator!=(const Vector<T> &lhs, const Vector<T> &rhs)
00805 {
00806     return !(lhs == rhs);
00807 }
00808
00815 template <typename T>
00816 void swap(Vector<T> &lhs, Vector<T> &rhs) noexcept
00817 {
00818     lhs.swap(rhs);
00819 }
00820
00828 template <typename T, typename U>
00829 constexpr size_t erase(Vector<T> &vector, const U &value)
00830 {
00831     auto first_removed = std::remove(vector.begin(), vector.end(), value);
00832     auto removed_elems_num = vector.end() - first_removed;
00833     vector.erase(first_removed, vector.end());
00834     return removed_elems_num;
00835 }
00836
00845 template <typename T, typename Pred>
00846 constexpr size_t erase_if(Vector<T> &vector, Pred pred)
00847 {
00848     auto first_removed = std::remove_if(vector.begin(), vector.end(), pred);
00849     auto removed_elems_num = vector.end() - first_removed;
00850     vector.erase(first_removed, vector.end());
00851     return removed_elems_num;
00852 }

```

7.25 src/zmogus.h Failo Nuoroda

```

#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include "klaidu_valdymas.h"

```

Klasės

- class [Zmogus](#)

7.26 zmogus.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005 #include <vector>
00006 #include <sstream>
00007
00008 #include "klaidu_valdymas.h"
00009
00010 class Zmogus
00011 {
00012 protected:
00013     std::string vardas_;
00014     std::string pavarde_;
00015
00016 public:
00017     Zmogus() : vardas_(""), pavarde_("") {}
00018     Zmogus(std::string vardas, std::string pavarde) : vardas_(vardas), pavarde_(pavarde) {}
00019     // kopijavimo konstr [Zmogus x(y) / Zmogus x = y]
00020     Zmogus(const Zmogus &kitas)
00021         : vardas_(kitas.vardas_),
00022           pavarde_(kitas.pavarde_) {}
00023     // perkélimo konstr [Zmogus x(std::move(y)) / Zmogus x = std::move(y)]
00024     Zmogus(Zmogus &&kitas)
00025         : vardas_(std::move(kitas.vardas_)),
00026           pavarde_(std::move(kitas.pavarde_)) {}
00027
00028     ~Zmogus() {}
00029
00030     // gavikai / getteriai
00031
00032     inline std::string vardas() const { return vardas_; }
00033     inline std::string pavarde() const { return pavarde_; }
00034
00035     // nustatytojai / setteriai
00036
00037     void nust_vardas(std::string vardas) { vardas_ = vardas; }
00038     void nust_pavarde(std::string pavarde) { pavarde_ = pavarde; }
00039
00040     virtual bool iverst_varda_pavarde(bool ar_iverstis_atsaukiama = false) = 0;
00041 };
00042
```


Rodyklė

- ~Studentas
 - Studentas, [15](#)
- ~Vector
 - Vector< T >, [25](#)
- ~Zmogus
 - Zmogus, [41](#)
- apsk_med
 - Studentas, [15](#)
- apsk_vid
 - Studentas, [15](#)
- ar_sk_ne_tarp_0_ir_10
 - investies_pagalb_fjos.cpp, [51](#)
 - investies_pagalb_fjos.h, [52](#)
- ar_sk_nedidesnis_uz_0
 - investies_pagalb_fjos.cpp, [51](#)
 - investies_pagalb_fjos.h, [52](#)
- ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil
 - investies_pagalb_fjos.cpp, [51](#)
 - investies_pagalb_fjos.h, [52](#)
- ar_sk_nera_1_2_3_4_5_6
 - investies_pagalb_fjos.cpp, [51](#)
 - investies_pagalb_fjos.h, [52](#)
- at
 - Vector< T >, [25](#), [26](#)
- back
 - Vector< T >, [26](#)
- begin
 - Vector< T >, [27](#)
- capacity
 - Vector< T >, [27](#)
- cbegin
 - Vector< T >, [27](#)
- cend
 - Vector< T >, [28](#)
- clear
 - Vector< T >, [28](#)
- const_iterator
 - Vector< T >, [22](#)
- const_pointer
 - Vector< T >, [22](#)
- const_reference
 - Vector< T >, [22](#)
- const_reverse_iterator
 - Vector< T >, [22](#)
- Container
 - konstantos_kt_klases.h, [57](#)
 - Studentas, [15](#)
- crbegin
 - Vector< T >, [28](#)
- crend
 - Vector< T >, [28](#)
- data
 - Vector< T >, [29](#)
- difference_type
 - Vector< T >, [22](#)
- duomenu_nuskaitymas
 - Programos_laikai, [11](#)
- duomenu_rikiavimas
 - Programos_laikai, [11](#)
- egzo_rezas
 - Studentas, [16](#)
- empty
 - Vector< T >, [29](#)
- end
 - Vector< T >, [29](#)
- erase
 - Vector< T >, [30](#), [31](#)
 - vector.h, [62](#)
- erase_if
 - vector.h, [63](#)
- failo_ivestis
 - ivestis.cpp, [54](#)
 - ivestis.h, [55](#)
- front
 - Vector< T >, [31](#)
- galut
 - Studentas, [16](#)
- generuota_ivestis
 - ivestis.cpp, [54](#)
 - ivestis.h, [55](#)
- insert
 - Vector< T >, [32](#), [33](#)
- isvesties_pagalb_fjos.cpp
 - pagal_mediana_did, [43](#)
 - pagal_mediana_maz, [43](#)
 - pagal_pavarde_did, [44](#)
 - pagal_pavarde_maz, [44](#)
 - pagal_varda_did, [44](#)
 - pagal_varda_maz, [44](#)
 - pagal_vidurki_did, [44](#)
 - pagal_vidurki_maz, [44](#)
 - ras_failo_paruosimas, [44](#)

- investies_pagalb_fjos.h
 - pagal_mediana_did, 45
 - pagal_mediana_maz, 45
 - pagal_pavarde_did, 45
 - pagal_pavarde_maz, 45
 - pagal_varda_did, 45
 - pagal_varda_maz, 46
 - pagal_vidurki_did, 46
 - pagal_vidurki_maz, 46
 - ras_failo_paruosimas, 46
 - rikiavimas, 46
 - stud_rikiavimas, 46
- investis
 - investis.cpp, 48
 - investis.h, 49
- investis.cpp
 - investis, 48
 - skirstoma_investis, 48
 - spausdinimas, 48
 - studentu_skirstymas, 48
 - visu_stud_duomenu_generavimo_investis, 48
- investis.h
 - investis, 49
 - skirstoma_investis, 49
 - spausdinimas, 49
 - studentu_skirstymas, 50
 - visu_stud_duomenu_generavimo_investis, 50
- iterator
 - Vector< T >, 22
- invest_egzo_reza
 - Studentas, 16
- invest_varda_pavarde
 - Studentas, 16
 - Zmogus, 41
- investies_klaidos_valdymas
 - klaidu_valdymas.cpp, 56
 - klaidu_valdymas.h, 56
- investies_pagalb_fjos.cpp
 - ar_sk_ne_tarp_0_ir_10, 51
 - ar_sk_nedidesnis_uz_0, 51
 - ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil, 51
 - ar_sk_nera_1_2_3_4_5_6, 51
 - natur_skaiciaus_investis, 51
 - skait_failo_paruosimas, 51
 - vardo_pavardes_investis, 52
- investies_pagalb_fjos.h
 - ar_sk_ne_tarp_0_ir_10, 52
 - ar_sk_nedidesnis_uz_0, 52
 - ar_sk_nedidesnis_uz_0_arba_didesnis_uz_10mil, 52
 - ar_sk_nera_1_2_3_4_5_6, 52
 - natur_skaiciaus_investis, 52
 - skait_failo_paruosimas, 53
 - vardo_pavardes_investis, 53
- investis.cpp
 - failo_investis, 54
 - generuota_investis, 54
 - misri_investis, 54
 - rank_investis, 54
 - visu_stud_duomenu_generavimo_investis, 54
- investis.h
 - failo_investis, 55
 - generuota_investis, 55
 - misri_investis, 55
 - rank_investis, 55
 - visu_stud_duomenu_generavimo_investis, 55
- klaidu_valdymas.cpp
 - investies_klaidos_valdymas, 56
- klaidu_valdymas.h
 - investies_klaidos_valdymas, 56
- konstantos_kt_klases.h
 - Container, 57
 - MAX_INT, 57
- main
 - pagr.cpp, 59
- MAX_INT
 - konstantos_kt_klases.h, 57
- misri_investis
 - investis.cpp, 54
 - investis.h, 55
- natur_skaiciaus_investis
 - investies_pagalb_fjos.cpp, 51
 - investies_pagalb_fjos.h, 52
- nust_egzo_reza
 - Studentas, 16
- nust_galutinio_tipa
 - Studentas, 17
- nust_pavarde
 - Zmogus, 41
- nust_varda
 - Zmogus, 42
- operator!=
 - vector.h, 63
- operator<<
 - Studentas, 19
 - studentas.cpp, 59
- operator>>
 - Studentas, 19
 - studentas.cpp, 59
- operator=
 - Studentas, 17
 - Vector< T >, 33
- operator==
 - vector.h, 64
- operator[]
 - Vector< T >, 34
- pagal_mediana_did
 - investies_pagalb_fjos.cpp, 43
 - investies_pagalb_fjos.h, 45
- pagal_mediana_maz
 - investies_pagalb_fjos.cpp, 43

isvesties_pagalb_fjos.h, [45](#)
 pagal_pavarde_did
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [45](#)
 pagal_pavarde_maz
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [45](#)
 pagal_varda_did
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [45](#)
 pagal_varda_maz
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [46](#)
 pagal_vidurki_did
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [46](#)
 pagal_vidurki_maz
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [46](#)
 pagr.cpp
 main, [59](#)
 pavarde
 Zmogus, [42](#)
 pavarde_
 Zmogus, [42](#)
 pazymiu_sk
 Studentas, [18](#)
 pointer
 Vector< T >, [23](#)
 pop_back
 Vector< T >, [35](#)
 prideti_pazymi
 Studentas, [18](#)
 Programos_laikai, [11](#)
 duomenu_nuskaitymas, [11](#)
 duomenu_rikiavimas, [11](#)
 spausd_laikus, [11](#)
 studentu_skirstymas, [11](#)
 visa_trukme_su_ivestim, [12](#)
 push_back
 Vector< T >, [35](#)
 rank_ivestis
 ivestis.cpp, [54](#)
 ivestis.h, [55](#)
 ras_failo_paruosimas
 isvesties_pagalb_fjos.cpp, [44](#)
 isvesties_pagalb_fjos.h, [46](#)
 rbegin
 Vector< T >, [36](#)
 reference
 Vector< T >, [23](#)
 rend
 Vector< T >, [36](#)
 reserve
 Vector< T >, [37](#)
 resize
 Vector< T >, [37](#)
 reverse_iterator
 Vector< T >, [23](#)
 rezas_med
 Studentas, [18](#)
 rezas_vid
 Studentas, [18](#)
 rikiavimas
 isvesties_pagalb_fjos.h, [46](#)
 shrink_to_fit
 Vector< T >, [38](#)
 size
 Vector< T >, [38](#)
 size_type
 Vector< T >, [23](#)
 skait_failo_paruosimas
 ivesties_pagalb_fjos.cpp, [51](#)
 ivesties_pagalb_fjos.h, [53](#)
 skirstoma_ivestis
 ivestis.cpp, [48](#)
 ivestis.h, [49](#)
 spausd_laikus
 Programos_laikai, [11](#)
 spausdinimas
 ivestis.cpp, [48](#)
 ivestis.h, [49](#)
 src_Directorijos_aprasas, [9](#)
 src/isvesties_pagalb_fjos.cpp, [43](#)
 src/isvesties_pagalb_fjos.h, [45](#), [47](#)
 src/ivestis.cpp, [47](#)
 src/ivestis.h, [49](#), [50](#)
 src/ivesties_pagalb_fjos.cpp, [50](#)
 src/ivesties_pagalb_fjos.h, [52](#), [53](#)
 src/ivestis.cpp, [53](#)
 src/ivestis.h, [55](#), [56](#)
 src/klaidu_valdymas.cpp, [56](#)
 src/klaidu_valdymas.h, [56](#), [57](#)
 src/konstantos_kt_klases.h, [57](#), [58](#)
 src/kt_klasiu_metodai.cpp, [58](#)
 src/pagr.cpp, [58](#)
 src/studentas.cpp, [59](#)
 src/studentas.h, [60](#)
 src/vector.h, [61](#), [65](#)
 src/zmogus.h, [72](#), [73](#)
 stud_rikiavimas
 isvesties_pagalb_fjos.h, [46](#)
 Studentas, [12](#)
 ~Studentas, [15](#)
 apsk_med, [15](#)
 apsk_vid, [15](#)
 Container, [15](#)
 egzo_rezas, [16](#)
 galut, [16](#)
 ivest_egzo_reza, [16](#)
 ivest_varda_pavarde, [16](#)
 nust_egzo_reza, [16](#)
 nust_galutinio_tipa, [17](#)
 operator<<, [19](#)
 operator>>, [19](#)
 operator=, [17](#)

- pazymiu_sk, 18
 - pridet_pazymi, 18
 - rezas_med, 18
 - rezas_vid, 18
 - Studentas, 14, 15
 - uzpildyt_pazymius_iki_min, 18
- studentas.cpp
 - operator<<, 59
 - operator>>, 59
- studentu_skirstymas
 - isvestis.cpp, 48
 - isvestis.h, 50
 - Programos_laikai, 11
- swap
 - Vector< T >, 38
 - vector.h, 64
- uzpildyt_pazymius_iki_min
 - Studentas, 18
- value_type
 - Vector< T >, 23
- vardas
 - Zmogus, 42
- vardas_
 - Zmogus, 42
- vardo_pavardes_ivestis
 - ivesties_pagalb_fjos.cpp, 52
 - ivesties_pagalb_fjos.h, 53
- Vector
 - Vector< T >, 23–25
- Vector< T >, 20
 - ~Vector, 25
 - at, 25, 26
 - back, 26
 - begin, 27
 - capacity, 27
 - cbegin, 27
 - cend, 28
 - clear, 28
 - const_iterator, 22
 - const_pointer, 22
 - const_reference, 22
 - const_reverse_iterator, 22
 - crbegin, 28
 - crend, 28
 - data, 29
 - difference_type, 22
 - empty, 29
 - end, 29
 - erase, 30, 31
 - front, 31
 - insert, 32, 33
 - iterator, 22
 - operator=, 33
 - operator[], 34
 - pointer, 23
 - pop_back, 35
 - push_back, 35
 - rbegin, 36
 - reference, 23
 - rend, 36
 - reserve, 37
 - resize, 37
 - reverse_iterator, 23
 - shrink_to_fit, 38
 - size, 38
 - size_type, 23
 - swap, 38
 - value_type, 23
 - Vector, 23–25
- vector.h
 - erase, 62
 - erase_if, 63
 - operator!=, 63
 - operator==, 64
 - swap, 64
- visa_trukme_su_ivestim
 - Programos_laikai, 12
- visu_stud_duomenu_generavimo_isvestis
 - isvestis.cpp, 48
 - isvestis.h, 50
- visu_stud_duomenu_generavimo_ivestis
 - ivestis.cpp, 54
 - ivestis.h, 55
- Zmogus, 39
 - ~Zmogus, 41
 - ivest_varda_pavarde, 41
 - nust_pavarde, 41
 - nust_varda, 42
 - pavarde, 42
 - pavarde_, 42
 - vardas, 42
 - vardas_, 42
 - Zmogus, 40